Technology
Arts Sciences
TH Köln

Master Thesis

# Design and Implementation of a Modular Architecture for the Transparent Integration of Containerized Execution Environments for Heterogeneous Open-Source Binaries in OneWare Studio

**Mert Torun**

**Institution:**
TH Köln – University of Applied Sciences
Faculty of Information, Media and Electrical Engineering

**Date:** 02.07.2026

**Supervisors:**
Prof. Dr. Tobias Krawutschke
M.Sc. Sebastian Wittlich

# Declaration on the Use of Aids

The statutory declaration (*Eidesstattliche Erklärung*) for this thesis is provided on the official cover sheet (*Deckblatt*). For transparency, I declare here the aids used in its preparation: in addition to the cited sources and standard software, I used AI-based assistance for language editing, formatting, and reviewing the manuscript. The design, implementation, evaluation, and all conclusions are my own; every part was authored, checked, and verified by me, and I take full responsibility for the correctness of the content.

# Abstract

Ensuring the reproducibility of build processes poses a significant challenge in embedded systems and Field-Programmable Gate Array (FPGA) development. The reliance on complex, platform-specific binaries and toolchains (e.g., Yosys, nextpnr) frequently leads to "Environment Drift," where builds succeed locally but fail in collaborative environments due to system inconsistencies.

To address these challenges, this thesis investigates three central Research Questions: how to establish a robust, cross-platform containerization architecture across heterogeneous host operating systems (RQ1), how to seamlessly integrate these containerized environments into a modern IDE to preserve a native user experience (RQ2), and what performance overhead is introduced by containerized execution and architecture emulation (RQ3). To resolve these questions, a modular software integration architecture is designed and implemented as a standalone extension for the *OneWare Studio* IDE. Using C# and .NET 10, a decoupled middleware is developed based on a **Hybrid Strategy Pattern**, enabling developers to switch seamlessly between high-performance native execution and hermetic containerized workflows at runtime.

The architecture resolves critical barriers to adoption through four technical contributions—three addressing the research questions directly, complemented by a reproducibility feature:

1. **Cross-Platform Interoperability (Addressing RQ1):** A dynamic User-ID injection strategy and abstract transport resolution ensure seamless operation across Windows, Linux, and macOS, specifically solving file-permission locking issues on Linux systems.

2. **Transparent Integration (Addressing RQ2):** Real-time stream multiplexing and log-level parsing transform raw container output into structured IDE feedback, preserving the native user experience.

3. **Performance Characterization (Addressing RQ3):** The system exposes a user-selectable image platform so that, on ARM64 Apple Silicon, an x86 toolchain image is executed through the host runtime's Rosetta 2 translation layer; the resulting execution overhead is characterized and decomposed.

4. **Temporal Reproducibility ("Time Machine"):** A dynamic tagging mechanism lets projects pin a specific toolchain image tag (optionally a content digest), providing a reproducible build environment over time.

The empirical evaluation characterizes the cost of the containerized path rather than asserting a single headline factor. Beyond raw containerization, the extension's strategy dispatch adds a fixed per-invocation cost on the order of 0.2 s and negligible marginal compute overhead, so its relative cost is dominated by a constant floor that amortizes as workload size grows. On Apple Silicon the toolchain image runs under Rosetta 2 emulation, so a native-versus-container comparison on that host would conflate containerization with instruction-set emulation; native `x86_64` hosts (Windows and Linux) reproduce the same fixed-floor structure without emulation, and the same-build native baseline on the Linux host—the one setting that isolates the container-versus-native factor—fixes it at $\alpha \approx 1$, a fixed per-invocation floor with no measurable per-instruction tax. Across the seven-workload FPGA corpus (iCE40 and ECP5 synthesis, place-and-route and bitstream packing, plus Icarus Verilog simulation) every workload executed to success through the hermetic container, so the system eliminates configuration friction and makes toolchain execution independent of the host's package environment, resolving cascading dependency conflicts.

**Keywords:** FPGA Toolchains; Docker Middleware; Hybrid Strategy Pattern; Reproducible Builds; Cross-Platform Architecture; .NET 10; Apple Silicon Emulation; User-ID Mapping; Version Pinning; Electronic Design Automation (EDA); OneWare Studio.

# Contents


**1 Introduction** **1**

1.1 Motivation and Problem Context . . . . . 1

1.2 Problem Statement . . . . . 1

1.3 Research Objectives . . . . . 2

1.4 Structure of the Thesis . . . . . 3

**2 Theoretical Background** **4**

2.1 Virtualization Paradigms in Software Engineering . . . . . 4

2.1.1 Hardware Virtualization (Hypervisors) . . . . . 5

2.1.2 Operating System-level Virtualization (Containers) . . . . . 6

2.1.3 Open Container Initiative (OCI) Standards . . . . . 6

2.2 The Docker Container Engine . . . . . 7

2.2.1 Kernel Namespaces . . . . . 7

2.2.2 System Call Filtering (Seccomp) . . . . . 9

2.2.3 Rootless Networking and TCP/IP Acceleration . . . . . 9

2.2.4 Control Groups (cgroups) . . . . . 10

2.2.5 Union File Systems (OverlayFS) . . . . . 11

2.2.6 Container Image Layer Caching and Layer Retrieval Protocols . . . 11

2.3 System Heterogeneity in EDA . . . . . 12

2.3.1 Instruction Set Architectures (ISA) and Binary Translation . . . . . 12

2.3.2 Operating System Interfaces (OSI) and Virtualization Environments 13

2.4 Comparative Analysis of Container Sandboxing Runtimes . . . . . 15

2.5 Modern FPGA Design Flows . . . 15
2.5.1 Synthesis and HDL Processing (Yosys/GHDL) . . . 15
2.5.2 Place and Route (nextpnr) . . . 16
2.5.3 Hardware Programming (openFPGALoader) . . . 16
2.6 The Docker Engine API . . . 17
2.6.1 IPC Transport Mechanisms . . . 17
2.6.2 API Protocol Operations . . . 18
2.6.3 Core Docker API Endpoints . . . 19
2.7 Software Architecture Patterns . . . 19
2.7.1 The Strategy Pattern . . . 19
2.7.2 The Adapter Pattern . . . 20
2.7.3 Dependency Injection . . . 20

**3 Context and State of the Art 21**
3.1 The Target Ecosystem: OneWare Studio . . . 21
3.2 Analysis of Existing Solutions . . . 21
3.2.1 Package Managers (FuseSoC) . . . 21
3.2.2 Functional Package Managers (Nix/Guix) . . . 22
3.2.3 Build System Generators (SiliconCompiler) . . . 22
3.2.4 Full-IDE Virtualization (VS Code DevContainers) . . . 23
3.2.5 IDE Extensions (TerosHDL) . . . 23
3.3 Comparative Evaluation of Virtualization Paradigms . . . 23
3.4 The Research Gap . . . 25

**4 Concept and Architecture 26**
4.1 The Hybrid Strategy Pattern . . . 26
4.2 Cross-Platform Transport and Namespace Mapping (RQ1) . . . 28
4.2.1 Dynamic Transport Resolution . . . 28
4.2.2 Rootless File Permissions and User-ID Injection . . . 30
4.3 State Machine for Container Lifecycle (Lazy Loading) . . . 31

4.3.1 Operational Execution Flow with Fallback Pathways . . . . . 32
4.4 Security Design Properties . . . . . 34

**5 Implementation** **36**
5.1 Design Decisions . . . . . 36
5.1.1 Choice of Technology Stack (.NET 10) . . . . . 36
5.1.2 Why Direct Docker Socket vs WSL2? . . . . . 37
5.2 Core Middleware (DockerExecutionStrategy) . . . . . 37
5.2.1 Architecture Selection (Rosetta 2 Platform Mapping) . . . . . 37
5.3 Low-Level Named Pipe Impersonation Override . . . . . 38
5.3.1 Why Reflection Is Required . . . . . 40
5.4 Log Stream Demultiplexing and Parsing . . . . . 41
5.4.1 Stream Multiplexing . . . . . 41
5.4.2 Regex Source Generators . . . . . 42
5.5 Recursive Segment-by-Segment Canonicalization . . . . . 42
5.6 Weak Process Lifecycle & Cancellation Propagation . . . . . 45
5.7 Argument-Vector Construction and Namespace Mapping (DockerCommand-Builder) . . . . . 48
5.8 Dynamic Registry Client and Tag Discovery . . . . . 49
5.9 OneWare Integration . . . . . 51

**6 Evaluation** **52**
6.1 Data Status and Threats to Validity . . . . . 52
6.2 Experimental Setup . . . . . 53
6.2.1 Hardware Testbed Configuration . . . . . 53
6.3 Methodology . . . . . 54
6.3.1 Harness Build and Timing Model . . . . . 55
6.3.2 Environmental Control and Hardware Isolation . . . . . 55
6.3.3 Execution Backends . . . . . 56
6.3.4 Timing and Statistical Protocol . . . . . 56
6.4 Quantitative Results (Performance) . . . . . 57

6.4.1 Extension Overhead on Machine 1 (macOS, Apple Silicon) . . . . . 57
6.4.2 Native Baseline and Overhead on Machine 2 (Linux, native x86_64) 58
6.4.3 Extension Overhead on Machine 3 (Windows, native x86_64) . . . 60
6.4.4 Structured Telemetry Performance . . . . . 61
6.4.5 Telemetry Scalability and Lock Contention Limits . . . . . 61
6.5 Analysis and Discussion . . . . . 62
6.5.1 Analysis of Performance Overhead . . . . . 62
6.5.2 The Native Baseline and the Version-Parity Gate . . . . . 63
6.5.3 Cross-Platform Output Determinism . . . . . 64
6.5.4 Virtualization Hypervisor and File Sharing Overhead . . . . . 66
6.5.5 Dynamic Binary Translation and Hardware TSO Memory Consistency 67
6.5.6 Argument-Contract Behavior under Command Chaining . . . . . 71
6.5.7 Failure Analysis: the Named-Pipe Attach Defect . . . . . 72
6.5.8 Reliability and Host Isolation . . . . . 74
6.5.9 External Validity (Generalizability) . . . . . 74
6.5.10 Construct Validity . . . . . 74

**7 Conclusion** **75**
7.1 Summary . . . . . 75
7.2 Evaluation Summary and Key Architectural Insights . . . . . 76
7.3 Future Work . . . . . 77

**References** **78**

**A Source Code Listings** **81**
A.1 High-Level Architectural Mapping . . . . . 81
A.2 Windows Named Pipe Privilege Escalation Override . . . . . 82
A.3 Path Resolution Security Verification . . . . . 83
A.4 Asynchronous Error Telemetry Channel Processing . . . . . 84

**B Cross-Platform Environment Manual** **86**

B.1 Installation . . . . . 86
B.2 Global Configuration Settings . . . . . 86
B.3 Runtime-Specific Configuration Manuals . . . . . 87
B.3.1 OrbStack (macOS) . . . . . 87
B.3.2 Docker Desktop (Windows and macOS) . . . . . 88
B.3.3 Podman (Rootless Linux Setup) . . . . . 88
B.4 Troubleshooting Guide . . . . . 89
B.4.1 Socket Permissions (Linux) . . . . . 89
B.4.2 Dangling Container Reap . . . . . 89

**C Benchmark and Telemetry Logs** **90**
C.1 Security Validation Output and Exceptions . . . . . 90
C.1.1 Shell Injection Command Blocking Exception . . . . . 90
C.2 JSON-Lines Telemetry Stress-Test Logs . . . . . 91

# List of Figures

4.1 Strategy Pattern for Tool Execution . . . . . 27
4.2 Dynamic Transport Protocol Resolution Flowchart . . . . . 29
4.3 State Machine of the 8-stage Lazy-Loading Container Lifecycle . . . . . 31
4.4 Flowchart of Ephemeral Container Execution Lifecycle . . . . . 33

5.1 Sequence of the reflection-based named-pipe stream-opener replacement . . 41
5.2 Docker Stream Multiplexing Frame . . . . . 41

6.1 Stage-2 virtualization and TSO path . . . . . 70

# List of Tables


2.1 Comparative Analysis of Virtualization Paradigms and System Latency . . 5

3.1 Comparative Matrix of Development Environment Virtualization and Hermetic Packaging Paradigms in EDA . . . . . 24

4.1 Detailed State Transition Table for the 8-Stage Container Lifecycle . . . . . 34

6.1 Extension overhead on Machine 1 (Apple M4 Max, macOS Tahoe 26.6, Docker 29.4.0). CLI and strategy backends only; $n = 30$ unless noted; 95 % Student-$t$ CIs. Ext. overhead is the strategy-to-CLI mean ratio. . . . . 58

6.2 Native baseline and overhead on Machine 2 (Intel Core Ultra 9 285H, Ubuntu 26.04, Docker 29.1.3, native `x86_64`). All three backends; $n = 30$ unless noted; means shown to 0.1 ms, with 95 % Student-$t$ CIs in the apparatus (`summary.csv`; all FPGA workloads CV $< 7$ %). $\Delta_{\text{cont}}$ is the additive container floor (CLI minus native); Cont. ov. the CLI-to-native ratio; Ext. ov. the strategy-to-CLI ratio. . . . . 59

6.3 Extension overhead on Machine 3 (Intel Core Ultra 7 155H, Windows 11, Docker 29.6.1, native `x86_64`). CLI and strategy backends only; $n = 30$ unless noted; 95 % Student-$t$ CIs. Ext. overhead is the strategy-to-CLI mean ratio. 60

6.4 Cross-platform artifact determinism: SHA-256 (first 12 hex digits) of each toolchain output on Machine 1 (macOS, ARM64 host, emulated `amd64` container), Machine 2 (Linux, native `amd64` container), and Machine 3 (Windows, native `amd64` container). . . . . 65

A.1 Mapping of Directory Structure to Key Architectural Classes . . . . . 81

B.1 Configuration settings keys defined in `ContainerExtensionModule.cs` . . . . . 87

# Listings


5.1 Settings-driven platform resolution . . . . . 38
5.2 Windows Named Pipe Impersonation Override . . . . . 38
5.3 Source-generated image-reference validation . . . . . 42
5.4 Path Canonicalization implementation . . . . . 42
5.5 Weak Process Lifecycle Wrapper . . . . . 46
5.6 Argument-vector construction (shell-less) . . . . . 48
5.7 Registry tag query and caching (simplified) . . . . . 49
A.1 Reflection-based Handler Interception in SecureNamedPipeCredentials . . . 82
A.2 Custom SecureStreamOpenerAsync with Identification Impersonation . . . 82
A.3 Recursive Symbolic Link Resolution (ResolveCanonicalInternal, condensed) 83
A.4 Asynchronous Channel Telemetry consumer loop . . . . . 84
C.1 Security Module Exception on Command Operator Check . . . . . 90
C.2 ContainerTelemetry Multi-Threaded Stress Test Output Log . . . . . 91

# List of Acronyms

**AOT** Ahead-of-Time (Compilation)

**API** Application Programming Interface

**CI** Continuous Integration

**CLI** Command Line Interface

**CLR** Common Language Runtime

**CPU** Central Processing Unit

**DAX** Direct Access

**DBT** Dynamic Binary Translation

**EDA** Electronic Design Automation

**FPGA** Field-Programmable Gate Array

**GID** Group Identifier

**GUI** Graphical User Interface

**HDL** Hardware Description Language

**I/O** Input/Output

**IDE** Integrated Development Environment

**IPC** Inter-Process Communication

**ISA** Instruction Set Architecture

**JIT** Just-in-Time (Compilation)

**OCI** Open Container Initiative

**OOM** Out of Memory

**OS** Operating System

**RISC** Reduced Instruction Set Computer

**RTL** Register-Transfer Level

**TSO** Total Store Ordering

**UID** User Identifier

**UX** User Experience

**VHDL** VHSIC Hardware Description Language

**VM** Virtual Machine

**WMO** Weak Memory Ordering

**WSL2** Windows Subsystem for Linux 2

# Chapter 1

# Introduction

The development of embedded systems and Field-Programmable Gate Arrays (FPGAs) has evolved significantly over the past decade. Modern hardware designs typically rely on extensive, often open-source toolchains—such as *Yosys* for synthesis [1], *nextpnr* for place-and-route [2], and *Verilator* [3] and *Icarus Verilog* for simulation. While these tools offer powerful capabilities, ensuring the reproducibility of build processes across heterogeneous development environments remains a critical challenge in Electronic Design Automation (EDA).

## 1.1 Motivation and Problem Context

In collaborative projects, a phenomenon widely referred to as "Environment Drift" poses a significant barrier to efficiency. A build process that succeeds on a developer's local machine frequently fails on a colleague's workstation or the Continuous Integration (CI) server due to subtle differences in system libraries, binary versions, or path configurations [4].

In the specific context of *OneWare Studio*, an Integrated Development Environment (IDE) aiming to unify FPGA development, the reliance on locally installed toolchains presents a bottleneck. Users are currently required to manually install and configure complex binaries that vary significantly between operating systems (Windows 11, macOS Tahoe 26.6, Ubuntu 26.04).

## 1.2 Problem Statement

Containerization technologies (e.g., Docker) offer a potential solution by encapsulating build tools into immutable artifacts ("Hermetic Builds"). However, standard Docker workflows typically require interaction via the Command Line Interface (CLI), which disrupts the seam-

less user experience expected from a GUI-based IDE. Specifically, three technical challenges inhibit widespread adoption:

1. **System Heterogeneity (RQ1):** The diversity of host platforms creates a compatibility matrix that is difficult to maintain natively. Integrating Docker requires handling platform-specific constraints, such as the divergence between Named Pipes on Windows and Unix Domain Sockets on Linux/macOS. Additionally, Linux file permissions often cause artifacts to be locked by the `root` user.
2. **Integration Gap (RQ2):** Developers must currently switch contexts between the IDE and a terminal to run container commands, breaking the "Flow" state. Real-time feedback, such as streamed build output in the IDE console, is lost.
3. **Performance Overhead (RQ3):** On Apple Silicon (ARM64), running the reference toolchain image—published for `x86_64` (`linux/amd64`) only—creates an instruction-set mismatch with the host. Executing it through the runtime's Rosetta 2 translation layer introduces a performance penalty that must be quantified to justify the trade-off.

## 1.3 Research Objectives

To address the described problem statement, this thesis follows a structured approach ranging from architectural design to user-experience validation. The research is guided by the following three research questions, ordered to mirror the development lifecycle. RQ1 and RQ3 are assessed quantitatively in the evaluation—for RQ3, the marginal cost of the integration layer is measured directly on all three hosts, the emulated-regime penalty is cited from the literature, and the native-versus-container factor is isolated only on a host with a same-build native baseline and is measured on the Linux host, the only one of the three that admits such a baseline (Machine 2). RQ2, concerned with the transparency and feel of the integration, is assessed qualitatively against the implemented behavior:

- **RQ1 (Architectural Foundation):** What architectural patterns ensure cross-platform compatibility for container orchestration across Windows, Linux, and macOS, specifically addressing low-level challenges like User-ID mapping and transport protocols?
- **RQ2 (Transparent Integration):** How can these containerized environments be integrated into a .NET-based IDE to provide a "Native User Experience," preserving essential features like real-time streamed log feedback and structured diagnostics?
- **RQ3 (Performance Viability):** What is the measurable performance overhead of this containerized approach compared to native execution, particularly when bridging architecture gaps (x86 on ARM64)?

## 1.4 Structure of the Thesis

The remainder of this thesis is organized as follows. Chapter 2 establishes the theoretical foundations of containerization, virtualization, and cross-architecture execution. Chapter 3 surveys the state of the art in reproducible EDA tooling and positions this work against existing approaches. Chapter 4 presents the modular architecture—the hybrid execution strategy, transport resolution, and container lifecycle—which Chapter 5 then realizes in detail. Chapter 6 reports the empirical evaluation of the integration overhead and cross-platform output determinism across the test machines, and Chapter 7 summarizes the contributions and outlines directions for future work.

# Chapter 2

# Theoretical Background

This chapter establishes the fundamental concepts and technologies required for the design of the containerized integration architecture. It provides a detailed analysis of virtualization paradigms, the internal mechanisms of the Docker engine, and the specific requirements of modern FPGA design flows.

## 2.1 Virtualization Paradigms in Software Engineering

Virtualization is the abstraction of computing resources to create isolated execution environments. In the context of this thesis, two primary paradigms are relevant: Hardware Virtualization (Hypervisors) and Operating System-level Virtualization (Containers) [5]. The choice between these paradigms determines the execution overhead, startup latency, and isolation characteristics of the compilation environment.

To quantitatively contrast these paradigms, Table 2.1 lists the key architectural properties and associated latency costs.

Table 2.1: Comparative Analysis of Virtualization Paradigms and System Latency

| Architectural Metric | Hardware Virtualization (VM) | OS-level Virtualization (Container) |
|---|---|---|
| **CPU State Transition** | Hypervisor Trap / VM-Exit (≈ 1000–3000 CPU cycles) | Native Syscall Trap (≈ 100–300 CPU cycles) |
| **Page Table Translation** | Two-Dimensional Paging (EPT/SLAT) | Native MMU Page Table Translation |
| **Kernel Instances** | Isolated Guest Kernel per VM | Shared Host Kernel |
| **Memory Allocation** | Statically Partitioned / Pre-allocated | Dynamically Allocated via Host |
| **Storage I/O Overhead** | Block-level Emulation / Virtual Disk | UnionFS / Native Bind Mounts |
| **Typical Boot Time** | Seconds to Minutes (10 s to 60 s) | Milliseconds (50 ms to 200 ms) |

Cycle and latency figures are representative order-of-magnitude values; [5] provides an application-level container-versus-VM performance comparison.

### 2.1.1 Hardware Virtualization (Hypervisors)

Hardware virtualization, managed by a Hypervisor (Type 1 running directly on bare metal, or Type 2 running on a host OS), creates fully isolated Virtual Machines (VMs). The guest operating system executes instruction streams within a deprivileged processor execution mode (e.g., Intel VMX non-root operation). Non-virtualizable sensitive instructions trigger a *VM-exit* event, forcing a context switch back to the host hypervisor for software emulation.

On macOS, the native kernel virtualization is exposed via the low-level `Hypervisor.framework` (which provides APIs for direct interaction with CPU virtualization features like VT-x/Intel VMX or Apple Silicon ARMv8 virtualization extensions) and the higher-level `Virtualization.framework` (which abstracts the bootloaders, virtual devices, and VM configuration). On Apple Silicon hosts, virtualization introduces unique scheduling dynamics. The host OS must map guest vCPU threads onto the physical asymmetrical cores: high-frequency Performance (P) cores and energy-efficient Efficiency (E) cores. The macOS thread scheduler manages these guest vCPU threads dynamically, but CPU pinning (affinity) is not natively supported by the framework. If guest threads are scheduled onto E-cores during intensive FPGA compilation, performance degrades significantly. Additionally, VM-exit penalties on ARM64 are caused by hypervisor traps for virtual timer access, interrupt handling, or generic stage-2 translation faults. While hardware-assisted translation (Two-Level Address Translation or Stage 2 Translation in ARMv8) reduces MMU translation latency by caching Guest Physical to Host Physical mappings in the TLB, TLB misses require walking two independent sets of

page tables, incurring a measurable performance tax on translation-heavy workloads compared to native execution.

Memory management in VMs introduces significant overhead due to Second Level Address Translation (SLAT), implemented as Extended Page Tables (EPT) on Intel CPUs or Nested Page Tables (NPT) on AMD CPUs. When the guest OS translates a guest virtual address to a guest physical address, the MMU must walk the two-dimensional guest-to-host page tables. This can double the page table walk latency in the worst-case cache-miss scenarios. Storage I/O similarly suffers from an emulation tax, where block device requests pass through multiple layers of virtualized queue adapters before reaching host storage. These factors result in a high resource overhead and make hypervisors unsuitable for the rapid, low-latency, iterative compile-and-run cycles required by modern Integrated Development Environments (IDEs).

### 2.1.2 Operating System-level Virtualization (Containers)

OS-level virtualization creates isolated user-space instances (containers) that share the host operating system's kernel. Rather than emulating hardware, it relies on host kernel features to partition resources, allowing processes inside the container to execute at native hardware speeds. System calls pass directly to the host kernel with normal kernel trap latencies, avoiding the VM-exit penalty.

For EDA toolchains, OS-level virtualization enables the concept of *Hermetic Builds*: environments that are completely self-contained and reproducible across developer machines. By locking the exact tool versions and system shared library hashes within an immutable container image, the container-based approach mitigates environmental divergence without incurring detrimental performance latency.

### 2.1.3 Open Container Initiative (OCI) Standards

To prevent vendor lock-in and ensure interoperability, modern containerization is governed by the Open Container Initiative (OCI) standards. The OCI Image Specification establishes a strict, content-addressable design [6]. An image is structured as a collection of tar files representing filesystem layers, coupled with a JSON manifest and a JSON configuration file. Content Addressability is achieved using cryptographic hashing (SHA-256). Every layer tarball, configuration file, and manifest is stored and indexed by its hash value. This allows the container engine to verify image integrity and reuse existing layers across different images, saving storage space and download bandwidth. The image manifest—itself a content-addressed JSON blob rather than a fixed filename—links these components together by listing the digests of the filesystem layers and the image configuration. The configuration file specifies execution metadata (environment variables, entrypoints, user accounts, and CPU architecture). During

runtime instantiation, the engine (e.g., `containerd`) downloads and verifies these layers. It then stacks them using a union filesystem like OverlayFS, presenting a unified read-only directory structure (the rootfs) to the runtime execution tool (e.g., `runc`).

The OCI Runtime Specification governs the execution environment [7]. The engine creates an isolated container environment using host kernel namespaces and cgroups as configured in the runtime's configuration file (`config.json`). The runtime defines standard lifecycle operations:

1. **create:** The container environment is created, namespaces are set up, and the init process is suspended before executing the entrypoint.
2. **start:** The suspended process is resumed, starting the target program.
3. **kill:** Sends a signal to the container processes.
4. **delete:** Cleans up the namespaces and resource limits.

The standardized OCI APIs allow container engines (such as Docker, Podman, or containerd) to swap underlying runtimes transparently, facilitating consistent execution environments across Linux, Windows, and macOS hosts.

## 2.2 The Docker Container Engine

Docker is the de-facto standard implementation of OS-level virtualization. It operates as a daemon (`dockerd`) that exposes a RESTful socket API to manage images, networks, volumes, and containers. The underlying container execution is driven by low-level kernel primitives, ensuring that containers run isolated processes with native scheduling and negligible hardware abstraction overhead.

### 2.2.1 Kernel Namespaces

Namespaces provide process-level resource isolation, partitioning the kernel resources so that a group of processes sees one set of resources while another group sees another.

1. **PID Namespace:** Virtualizes process IDs, meaning the first process spawned inside the container becomes PID 1. This prevents containerized processes from viewing or signaling processes on the host or other containers.
2. **Mount Namespace (mnt):** Isolates filesystem mount points. The container sees a custom directory layout with its own root directory (`/`) independent of the host. Crucially, mount propagation flags control how mounts inside or outside the namespace affect each other:

- **shared:** Mounts propagate in both directions. A mount inside the container propagates to the host, and vice versa.
- **slave:** Mounts propagate only from the host to the container. If the host mounts a new volume under a shared mount, the container sees it, but mounts created inside the container do not reflect back on the host.
- **private:** Mounts do not propagate in either direction. The containerized mount environment is completely decoupled from the host.
- **rprivate/rshared/rslave:** Recursive versions of the above flags, which apply the propagation properties to all sub-mounts within the directory tree. Docker bind mounts default to `rprivate` (recursive-private, no propagation in either direction). Correct propagation settings are critical for mounting active design workspaces, preventing host-level modifications from breaking container compiler contexts.

3. **User Namespace (user):** Maps user and group IDs (UID/GID) inside the container to a different set of IDs on the host. This is a critical security feature: a process running as root (UID 0) inside the container can be mapped to an unprivileged user (e.g., UID 10001) on the host, preventing host takeover if a container breakout occurs.

   In modern rootless container architectures, user namespaces enable unprivileged users to create and run containers safely without requiring root access on the host operating system [8]. When an unprivileged user starts a rootless container, the container engine uses helper binaries like `newuidmap` and `newgidmap` to consult the host's configuration files, namely `/etc/subuid` and `/etc/subgid`. These files define ranges of subordinate user and group IDs allocated to the host user. The engine maps the container's internal root user (UID 0) to the host user's actual UID (e.g., UID 1000), while mapping internal UIDs 1 to 65 536 to the allocated range of subordinate UIDs on the host (e.g., UIDs 100 000 to 165 535). Thus, while the containerized process runs with root privileges (such as `CAP_SYS_ADMIN` or `CAP_NET_ADMIN`) within its own isolated namespace, it possesses only the unprivileged host user's permissions on the host system, mitigating privilege escalation attacks.

   The relevant consequence for this thesis is qualitative. Because container-internal IDs are translated to host IDs through these mapping ranges, a process that is root inside the namespace holds only the unprivileged host user's permissions outside it, so a container escape yields no host privilege. This is what makes the extension's host UID/GID injection meaningful—artifacts written by the containerized toolchain are owned by the invoking developer rather than by `root`—and is revisited as a design property in Chapter 4. The detailed range-mapping algebra enforced by `newuidmap`/`newgidmap` is part of the container runtime, not of this work.

4. **Network (net) and IPC Namespaces:** Isolate network interfaces, routing tables,

and Inter-Process Communication channels (such as shared memory segments and semaphores), ensuring network and socket containment.

### 2.2.2 System Call Filtering (Seccomp)

Beyond namespaces, container runtimes apply a default *seccomp* (secure computing) profile that restricts which system calls a containerized process may issue, blocking administrative calls such as those that change system time, load kernel modules, or manipulate namespaces. This is a property of the runtime's default configuration rather than something the extension programs itself: the extension relies on the runtime defaults and, at container creation, additionally drops all Linux capabilities and sets `no-new-privileges` (Chapter 4). A detailed treatment of the seccomp-BPF filter internals is outside the scope of this thesis.

### 2.2.3 Rootless Networking and TCP/IP Acceleration

While rootless containers significantly harden the security boundary, isolating the network namespace without administrative privileges introduces severe network performance bottlenecks. Because an unprivileged user cannot create kernel-level virtual ethernet (`veth`) interfaces or configure host-level routing tables, rootless container engines traditionally rely on user-space network translation utilities like `slirp4netns`. Under this scheme, every TCP/IP packet generated inside the container's network namespace must be copied from kernel space into user space, translated by the `slirp4netns` daemon, and then reinjected back into the host's network stack via unprivileged socket API calls. This constant context-switching and user-space packet copying inflicts substantial CPU overhead and reduces network throughput, which can severely slow down operations such as pulling multi-gigabyte compiler images or contacting license servers.

To accelerate communication in unprivileged environments, Matsumoto and Suda proposed `bypass4netns` [9]. This technique intercepts socket-related system calls (e.g., `socket`, `connect`, `bind`) made by the containerized process using the Linux secure computing (seccomp) facility with Berkeley Packet Filter (BPF) filters. When a socket call is intercepted, the seccomp filter pauses the container process and notifies a tracer daemon running in the host's native network namespace. The tracer daemon instantiates a native socket directly on the host and issues the `SECCOMP_IOCTL_NOTIF_ADDFD` `ioctl` operation on the seccomp notification file descriptor to inject the corresponding file descriptor back into the containerized process's file descriptor table. This allows network traffic to bypass the unprivileged network namespace and flow directly through the host's high-performance kernel TCP/IP stack. Empirical evaluations of `bypass4netns` demonstrate over a 30 × increase in TCP throughput, significantly improving sustained bandwidth. However, this seccomp BPF interception path introduces a user-kernel-

user context-switch latency tax. Every `socket()`, `bind()`, and `connect()` system call requires trapping to the kernel, notifying the host tracer daemon in user space, injecting the file descriptor, and returning. Consequently, while sustained streaming throughput is greatly improved, connection-setup latency is degraded. For EDA toolchains that execute rapid, short-lived network queries (e.g., license server heartbeats or polling), this seccomp redirection overhead can introduce noticeable latency penalties compared to native execution.

### 2.2.4 Control Groups (cgroups)

While namespaces isolate process views, Control Groups (cgroups) manage, meter, and limit the physical resources consumed by those processes. The Linux kernel has transitioned from cgroups v1 to cgroups v2, which introduces a unified resource hierarchy. Under cgroups v1, controllers were completely independent. A process could belong to one group for memory constraints and a different group for CPU throttling. This led to complex synchronization issues and high kernel management overhead. A critical issue in cgroups v1 was the inability to coordinate memory and Block I/O controllers for buffered I/O writebacks. The kernel's page cache writeback daemon (which runs as a background process) could not determine which cgroup owned a dirty page. As a result, writeback I/O throttling could not be applied correctly to the process that wrote the data, allowing containers to bypass configured I/O limits. Furthermore, cgroups v1 allowed individual threads of a single process to be placed in different cgroups, complicating resource allocation and tracking.

Cgroups v2 establishes a single unified tree structure where every resource controller (CPU, memory, I/O, PID count) is attached to a single group hierarchy:

- **Single-Hierarchy Rule:** A process can only reside in a single cgroup node. All controllers apply to the same process group, enabling cleaner resource management and cross-resource controllers (such as the Out-of-Memory (OOM) killer terminating an entire container group rather than a single random leaf process).
- **Process-Granularity:** Resource allocation is restricted to the process level. All threads of a process must belong to the same cgroup, preventing thread grouping inconsistencies.
- **Unified Writeback Tracking:** The memory and I/O controllers cooperate, allowing the kernel to correctly attribute buffered I/O writebacks to the responsible cgroup and apply disk I/O limits.
- **Improved Resource Control Files:** Controllers use unified control files: `memory.max` (hard limit, triggers OOM killer when exceeded), `memory.high` (throttle limit, slows down memory allocation when exceeded to avoid hard OOM exits), `cpu.max` (defines the

CPU quota, e.g., `100000 100000` for 1 core or `400000 100000` for 4 cores), and `cpu.weight` (replaces v1's CPU shares, using a standardized scale of 1 to 10 000).

Integrating resource constraints ensures that intensive synthesis operations do not consume all host resources, which would freeze the developer's workstation or IDE interface.

### 2.2.5 Union File Systems (OverlayFS)

Docker images are constructed from read-only layers stacked using a Union File System (typically OverlayFS). OverlayFS merges two directories on a single host into a unified view. The bottom directory (*lowerdir*) represents the immutable base image layer, while the top directory (*upperdir*) contains the writable container layer. When a container writes to a file that exists in the base image, OverlayFS performs a *Copy-on-Write* (CoW) operation: it copies the file from the read-only lower directory to the writable upper directory before executing the write. This mechanism guarantees:

- **Efficiency:** Read-only layers are shared among all running container instances, saving disk space and memory cache.
- **Reproducibility:** The base layers remain untouched, ensuring that every time a design tool compiles, it starts from a clean, identical filesystem state.

### 2.2.6 Container Image Layer Caching and Layer Retrieval Protocols

To minimize network overhead and startup latency, container engines employ sophisticated layer caching and retrieval protocols governed by the OCI Distribution Specification. An image layer is a compressed tarball identified by its cryptographic SHA-256 digest, which enforces content-addressable storage (CAS).

When a container engine is instructed to retrieve an image (e.g., via the Docker Engine API), it performs a multi-step negotiation protocol with the registry:

1. **Manifest Retrieval:** The client requests the image manifest by its repository and reference (tag or digest). The registry returns a JSON document specifying the schema version, config digest, and an ordered list of layer descriptors containing their media types, digests, and byte sizes.
2. **Existence Check (Blob Probing):** The client probes its local content store to determine which layer digests are already present. This constitutes the local cache layer lookup. For layers already present, the client skips downloading entirely, achieving $O(1)$ cache hit resolution.

3. **Parallel Layer Fetching:** For cache misses, the client initiates concurrent HTTP GET requests to the registry's blob endpoint (`/v2/<name>/blobs/<digest>`). To optimize network throughput, layers are downloaded in parallel.

4. **Decompression and Stacking:** Downloaded tarballs are verified against their SHA-256 digests, decompressed, and registered in the local graph driver (e.g., OverlayFS).

In the context of EDA toolchains, where toolchain image sizes can range from 2 GB to 15 GB due to extensive commercial compilers and libraries, efficient layer reuse is critical. By structuring Dockerfiles with stable system dependencies (e.g., base compiler libraries, glibc, Python runtimes) at the bottom and frequently changing plugin configurations or script templates at the top, developers can maximize layer sharing. Under this configuration, when a user switches between different tool versions (e.g., Yosys v0.38 to v0.39), only the thin top layer representing the tool binary is downloaded, while the massive base dependency layers remain cached locally.

## 2.3 System Heterogeneity in EDA

EDA toolchains present extreme OS and architecture heterogeneity, complicating deployment in cross-platform environments.

### 2.3.1 Instruction Set Architectures (ISA) and Binary Translation

Most legacy and commercial FPGA synthesis and layout tools are compiled exclusively for the **x86_64** (AMD64) architecture. However, contemporary development hardware is shifting toward ARM64, as seen with Apple Silicon and ARM-based Windows platforms. Running x86_64 binaries on ARM64 hosts requires dynamic or ahead-of-time binary translation [10].

A core challenge in cross-architecture binary translation lies in bridging divergent hardware Memory Consistency Models (MCM). Memory consistency models dictate the rules for how memory writes from one CPU core become visible to other cores. The x86 architecture enforces a strong memory consistency model known as Total Store Ordering (TSO), which guarantees that memory writes become globally visible in program order; only the store-to-load order is relaxed by store buffering. Software developers and compilers targeting x86 rely on TSO to synchronize multi-threaded processes without injecting explicit hardware memory barriers. Conversely, ARM64 and RISC-V employ a Weak Memory Ordering (WMO) model, allowing aggressive hardware instruction reordering to maximize pipeline efficiency. When executing an x86 binary on a WMO host like ARM64, standard dynamic binary translation (DBT) engines (such as QEMU) must inject expensive software memory barriers (`DMB` or `DSB`)

after memory operations to simulate TSO constraints, which serializes execution and can degrade multi-threaded performance substantially [11], [12], [13].

On macOS, Apple implements **Rosetta 2** [14], which translates x86_64 instructions into equivalent ARM64 instruction sequences. Rosetta 2 compiles x86_64 user-space binaries during their initial load (Ahead-of-Time translation) and caches the resulting ARM64 code on disk. If dynamic instruction translation is needed (such as JIT compilation), Rosetta 2 performs Just-in-Time compilation at runtime. Rosetta 2 leverages custom hardware extensions in Apple CPUs, such as a hardware-level switch that enables TSO memory consistency directly on the ARM cores, which matches the x86 memory model and avoids expensive memory fence emulations [15]. Despite these hardware optimizations, binary translation imposes a performance cost, referred to hereafter as the binary translation overhead. This overhead is caused by instruction expansion (ARM64 instructions are fixed-width and simpler than complex x86 instructions), page table alignment translation (macOS defaults to 16 kB pages while Linux/x86_64 expects 4 kB pages), and system call mapping overheads.

### 2.3.2 Operating System Interfaces (OSI) and Virtualization Environments

Because Docker relies on Linux kernel primitives (namespaces, cgroups), running containers on macOS or Windows requires a virtualized Linux kernel. This kernel runs inside a lightweight VM managed by hypervisor frameworks:

- **macOS (OrbStack or Docker Desktop):** Uses macOS's native `Hypervisor.framework` to boot a minimal Linux kernel in a deprivileged VM.
- **Windows (WSL2):** Uses the Hyper-V hypervisor to run a Microsoft-optimized Linux utility VM.

In these virtualized setups, accessing host files from within the container requires a network- or filesystem-level sharing bridge, which has significant performance implications [16]. The performance of these bridges is a critical bottleneck for I/O-intensive EDA tasks like compiling thousands of VHDL/Verilog source files:

- **9pfs Protocol:** Early hypervisor implementations utilized the 9pfs (Plan 9 Filesystem) protocol to share directories. Because 9pfs acts as a remote network filesystem, every filesystem query requires packet serialization, transmission over a virtual network link, hypervisor context switching, and deserialization. When subjected to the highly parallelized, random small-file access patterns of HDL compilation, 9pfs exhibits catastrophic I/O operations per second (IOPS) degradation.
- **gRPC-FUSE:** Transports filesystem requests by translating OS-level VFS (Virtual File System) operations into gRPC messages sent over a virtual socket. This protocol is

highly abstract and incurs severe IPC serialization overheads, leading to high latency and low throughput on small file operations. Every metadata check (`stat`) or read request requires a full user-to-kernel context switch in the guest, serialization, socket transport, and context switch on the host, leading to latency overheads of several milliseconds per file operation.

- **VirtioFS:** A paravirtualized file sharing protocol designed specifically for virtualization [17]. It implements a shared-memory region between the guest VM and the host, allowing the VM to bypass socket serialization and access host file metadata and data directly. VirtioFS significantly reduces latency and file attribute caching costs compared to gRPC-FUSE, making it the preferred choice for mounting active design workspaces.

Furthermore, VirtioFS supports various caching modes (such as `none`, `metadata`, and `always`) to tune consistency and performance. A crucial architectural optimization in VirtioFS is **Direct Access (DAX)**. In traditional hypervisor file sharing, files are cached twice: once in the host kernel's page cache and once in the guest kernel's page cache, which wastes RAM and CPU cycles during cache sync operations. With DAX support, the host's page cache is mapped directly into the guest VM's physical address space. When a containerized compiler reads a source file, the guest kernel accesses the host's page cache memory directly, bypassing guest-level caching and eliminating double caching overheads.

Despite the DAX optimization, shared directory structures still suffer from metadata traversal latency (e.g., path resolution and permission checking). In high-density serverless and microVM environments, the storage interface design dictates startup concurrency and resource utilization. In runtime environments like RunD [18], directory-sharing mechanisms are bypassed in favor of a block-level device overlay (`virtio-blk`) configured via device-mapper. The guest VM mounts an `ext4` filesystem directly on the block device, eliminating all hypervisor-level metadata traversal during execution. Empirical measurements from RunD show that leveraging `virtio-blk` along with optimized page cache sharing allows launching over 200 secure microVM containers per second, accommodating up to 2500 concurrent instances on a single physical host node with minimal memory usage. Similar container-on-microkernel architectures, such as MettEagle [19], demonstrate that microkernel-isolated container runtimes can achieve near-native performance while restricting the host kernel attack surface.

However, this block-level overlay approach exposes a critical system concurrency limitation. Because standard filesystems like `ext4` are not cluster-aware (unlike clustered filesystems such as OCFS2), a raw block device cannot be concurrently mounted read/write by both the host IDE and the guest VM without immediate and catastrophic metadata corruption. Thus, while `virtio-blk` overlays are highly effective in isolated, read-only serverless worker nodes, they are unusable for interactive IDE environments where the developer on the host must dynamically write and read the workspace files. For these mixed-read/write environments, paravirtualized

shared filesystems such as VirtioFS, or network filesystems such as 9pfs, remain systemically mandatory to coordinate concurrent file locking and maintain cache consistency.

## 2.4 Comparative Analysis of Container Sandboxing Runtimes

For secure execution of untrusted EDA code, the runtime sandbox choice determines the kernel attack surface. This section compares three runtime solutions:

1. **runc:** The default OCI reference runtime written in Go. It executes container processes directly as child processes of the host kernel namespaces. It provides native execution speed with zero virtualization overhead but shares the host kernel interface. If a containerized synthesis tool exploits a kernel system call vulnerability, it can break out to the host.

2. **crun:** A fast, low-memory OCI runtime written in C. It performs identical namespace isolation as runc but has lower startup latency (on the order of single-digit versus tens of milliseconds). However, it shares the same host kernel attack surface.

3. **gVisor:** A secure sandbox runtime developed by Google that implements a user-space kernel (called the "Sentry") written in Go. The Sentry intercepts all system calls made by the container and handles them in user space, exposing only a tiny, hardened subset of system calls to the host kernel; filesystem access outside the sandbox is delegated to a separate file-proxy process (the "Gofer"). While gVisor dramatically reduces host kernel exploit vulnerability, it imposes a high system-call translation tax (several-fold slower on I/O-intensive builds), making it unsuitable for high-throughput FPGA synthesis.

## 2.5 Modern FPGA Design Flows

The FPGA compilation process, leading to "Bitstream Generation," is a multi-stage pipeline where each stage depends on distinct, highly specialized command-line tools. Maintaining this pipeline is difficult due to complex dependency chains and libraries.

### 2.5.1 Synthesis and HDL Processing (Yosys/GHDL)

Synthesis is the process of translating high-level Hardware Description Languages (VHDL/Verilog) into a structural netlist of technology-mapped logic gates:

- **GHDL:** GHDL is an open-source simulator that executes VHDL designs directly [20]. It is built against one of three code-generation backends (the built-in mcode JIT, LLVM, or GCC); the LLVM and GCC builds are tightly coupled to host compiler toolchains.
- **Yosys:** A framework for Verilog RTL synthesis. For VHDL synthesis, Yosys relies on GHDL integrations (e.g., via the `ghdl-yosys-plugin`).

These tools depend on specific versions of shared libraries like `libboost_system`, `libboost_thread`, and exact glibc runtimes. Divergent shared library dependencies frequently precipitate severe dependency resolution cascades.

### 2.5.2 Place and Route (nextpnr)

Place and Route (P&R) maps the synthesized netlist onto the physical logic blocks, routing resources, and I/O pins of a target FPGA architecture. This stage is highly computational. nextpnr links against specific versions of the Boost C++ shared libraries (e.g., `libboost_program_options`, `libboost_iostreams`, `libboost_thread`). If the host system updates its package manager (e.g., via macOS Homebrew or Windows Chocolatey), the installed Boost version may upgrade from v1.82 to v1.85. This causes nextpnr to fail with a dynamic linker error (`dyld: Library not loaded`), a classic example of package drift breaking toolchain execution.

### 2.5.3 Hardware Programming (openFPGALoader)

Once the bitstream is generated, it must be loaded onto the physical FPGA hardware. This is performed using utility software like `openFPGALoader` or `iceprog`, which interact with host USB-to-UART/FIFO controllers (typically FTDI chips). These utilities depend on low-level USB drivers like `libusb` and `libftdi`. Containerizing these tools requires not only runtime isolation but also raw host USB device path forwarding into the guest VM space, which complicates virtualized environments.

## 2.6 The Docker Engine API

The Docker daemon exposes its functionality via a structured, versioned RESTful HTTP API, typically accessed through a local IPC channel. This interface allows external programs, such as OneWare Studio, to manage the container lifecycle programmatically.

### 2.6.1 IPC Transport Mechanisms

Depending on the host operating system, the API client must resolve and establish a socket connection. Windows Named Pipes and Unix Domain Sockets are standard local IPC mechanisms [21]:

- **Unix Domain Sockets (UDS):** On macOS and Linux, the standard communication channel is a local IPC socket located at `/var/run/docker.sock`. Sockets are lightweight and do not cross the TCP/IP stack, avoiding network overhead. They use standard file-like descriptor operations (`socket(AF_UNIX, SOCK_STREAM)`).
- **Named Pipes:** On Windows hosts, the API is exposed via a Named Pipe at the path `\\.\pipe\docker_engine`. Named Pipes are part of the Windows NT kernel namespace and support asynchronous I/O and security descriptor configurations.

### 2.6.2 API Protocol Operations

Communication over the HTTP stream consists of JSON payloads for container creation and raw TCP connections upgraded to hijacked stdin/stdout streams for interactive execution. The client translates high-level task requests into the corresponding JSON payloads.

### 2.6.3 Core Docker API Endpoints

The middleware orchestrates containers using four core REST endpoints from the Docker API [22]:

1. **Create Container (`POST /containers/create`):** Sends a JSON payload defining the configuration (image, entrypoint, environment variables, volumes, and cgroup limits). The daemon returns a JSON payload containing the container's unique 64-character hexadecimal ID.
2. **Attach to Streams (`POST /containers/[id]/attach`):** Establishes a persistent, raw TCP/pipe connection to stream standard output and standard error. The daemon multiplexes stdout and stderr over this stream using an 8-byte frame header (detailed in Chapter 5).
3. **Start Container (`POST /containers/[id]/start`):** Instantiates the container: creates its namespaces, mounts the UnionFS filesystem layers, and executes the entrypoint.
4. **Wait Container (`POST /containers/[id]/wait`):** A blocking call that returns a JSON response containing the container process's final exit code when execution terminates.

This programmatic interface ensures that the IDE can monitor build progress, capture exit codes, and process stdout/stderr streams with low latency, without resorting to sub-process spawning of command-line tools.

## 2.7 Software Architecture Patterns

Integrating containerized toolchains into a modern IDE without sacrificing modularity requires applying robust software architecture patterns that govern enterprise application structures [23]. The OneWare Container Extension leverages three main patterns:

### 2.7.1 The Strategy Pattern

The Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. In this implementation, the strategy represents the execution engine. By defining the `IToolExecutionStrategy` interface, the IDE decouples the compiler modules (which generate synthesis or simulation commands) from the physical execution layer (which runs those commands). The compiler is unaware of whether a command is executed natively on the host shell or inside a container sandbox. This design allows for runtime flexibility, enabling the IDE to dynamically switch strategies based on daemon availability and user settings.

### 2.7.2 The Adapter Pattern

The Adapter Pattern converts the interface of a class into another interface clients expect. OneWare Studio's core expects standard OS processes (`System.Diagnostics.Process`) to monitor execution, manage cancellation tokens, and read streams. However, container execution is carried out asynchronously via HTTP requests over a Unix socket or Named Pipe. To bridge this gap, the middleware implements the Adapter Pattern in `StartWeakProcess`. It wraps the container's socket connection inside a dummy OS process handle, translating native process operations (such as `Kill()`) into Docker API lifecycle calls—a graceful `POST /containers/[id]/stop` with a short kill-timeout, followed by a `DELETE /containers/[id]` teardown—ensuring compatibility with existing IDE code.

### 2.7.3 Dependency Injection

To achieve loose coupling, the architecture utilizes Dependency Injection (DI) to manage service lifetimes and configuration settings. Services such as `DockerExecutionStrategy` and the diagnostics view model are registered as Singletons within the application's dependency container, and the host-provided `ISettingsService` is consumed as a singleton; the telemetry component is a process-wide static utility (a JSON Lines writer) rather than a DI-registered service. This ensures that a single instance coordinates container lifecycle operations and prevents socket exhaustion, while configuration changes propagate dynamically to all dependent modules.

# Chapter 3

# Context and State of the Art

## 3.1 The Target Ecosystem: OneWare Studio

OneWare Studio is a modern, cross-platform Integrated Development Environment (IDE) for FPGA development, built on the .NET 10 framework and Avalonia UI [24]. **Reproducibility** across heterogeneous machines—student laptops and CI servers alike—is a recognized requirement for collaborative and educational EDA workflows [25]. Unlike textual editors (e.g., Vim, Sublime), OneWare provides rich semantic features (LSP, Diagramming) that require low-latency interaction with the underlying toolchain.

## 3.2 Analysis of Existing Solutions

To contextualize the proposed architecture, we analyze five existing approaches to environment management in EDA: Package Managers (FuseSoC), Functional Package Managers (Nix/Guix), Build System Generators (SiliconCompiler), Full-IDE Virtualization (DevContainers), and toolchain-managing IDE Extensions (TerosHDL).

### 3.2.1 Package Managers (FuseSoC)

*FuseSoC* is a package manager and build abstraction tool for FPGA designs. It focuses on dependency resolution for *HDL code* (libraries, IP cores).

- **Strength:** Excellent for managing project code dependencies.
- **Weakness:** It assumes the existence of the underlying toolchain (Yosys, Vivado) on the host path. It does not solve the "Missing Binary" problem or handle distinct tool

versions for different projects (e.g., Project A needs Yosys 0.9, Project B needs Yosys 0.33).

### 3.2.2 Functional Package Managers (Nix/Guix)

Functional Package Managers (FPMs), such as Nix and GNU Guix, represent a paradigm shift in environment reproducibility by modeling software builds as pure, mathematical functions [4].

- **Mechanism:** Under this paradigm, every build input—including source files, compilation flags, compiler versions, and the transitive dependency graph—is cryptographically hashed to produce a unique, immutable store path (such as `/nix/store/h1a2b3c4...-yosys-0.9`). Divergent environments coexist without interference, eliminating dependency conflicts at the filesystem level. GNU Guix extends this by using G-expressions in Scheme for deeply integrated code staging.
- **Reproducibility rate:** Large-scale empirical studies on the nixpkgs repository evaluated over 709 816 historical packages, validating a rebuildability rate exceeding 99 % and a bitwise reproducibility rate between 69 % and 91 % [26]. Scale monitoring is achieved via decentralized architectures like Lila [27].
- **Critique (Process and Network Exposure):** Because FPMs operate exclusively at the filesystem level, they lack runtime namespace containment. Processes execute directly on the host, sharing the host's PID tree, network devices, and user directories, which poses substantial security risks when building untrusted EDA scripts. Furthermore, many legacy or commercial EDA tools rely on hardcoded paths (e.g., `/opt/eda`) that violate the pure store model, requiring complex binary patching (e.g., via `patchelf`) to run.

### 3.2.3 Build System Generators (SiliconCompiler)

Tools like *SiliconCompiler* [28] utilize Python scripts to abstract the toolchain complexity.

- **Mechanism:** Input HDL → Python Script → Execution Graph → Bitstream.
- **Critique (Execution Opacity):** They operate as blocking CLI wrappers. These abstractions enforce a rigid execution graph and obscure underlying compilation telemetry until termination. This opaque execution model degrades the IDE experience (RQ2), as the IDE cannot easily parse real-time errors or interject during the build process.

### 3.2.4 Full-IDE Virtualization (VS Code DevContainers)

Microsoft's VS Code *DevContainers* extension allows developers to open a project inside a Docker container.

- **Mechanism:** The entire IDE "Backend Server" runs inside the container alongside the tools. The UI runs on the host and communicates via a socket.
- **Critique (Resource Heaviness):** This paradigm imposes a rigid, resource-intensive virtualization boundary, necessitating the provisioning of a complete operating system user-space per project. For FPGA development, this often means routing X11/Wayland traffic for waveform viewers (GTKWave), which is fragile and non-performant on macOS/Windows.

### 3.2.5 IDE Extensions (TerosHDL)

*TerosHDL* [29] is a VS Code extension that attempts to manage toolchains.

- **Weakness:** It relies on static provisioning (downloading pre-built binaries) and platform-dependent shell scripts. This often breaks on Windows (Git Bash vs PowerShell) and lacks the hermetic guarantees of containers (RQ1).

## 3.3 Comparative Evaluation of Virtualization Paradigms

To systematically evaluate the state of the art, Table 3.1 consolidates these approaches into four representative virtualization and packaging paradigms—the host-native tools (FuseSoC, SiliconCompiler, TerosHDL) share a single column—and compares them against the constraints of EDA toolchains.

As shown, Nix and Guix guarantee immutable reproducibility and zero-latency filesystem access, but their lack of runtime network and process namespace containment leaves the host exposed to untrusted build scripts. Full-IDE virtualization (e.g., DevContainers) offers strong isolation for both language server and compiler, but incurs heavy host resource overhead and filesystem mapping latencies over hypervisor channels like 9pfs or VirtioFS. Tool-Virtualization Middleware (such as the OneWare Container Extension) bridges this gap by isolating only the per-invocation tool process within the container runtime, maintaining the responsiveness of the host-native IDE UI with near-native bind mount performance, and hardening the container through capability dropping (`--cap-drop ALL`), `no-new-privileges`, and a configurable network mode, under the runtime's default seccomp profile.

Table 3.1: Comparative Matrix of Development Environment Virtualization and Hermetic Packaging Paradigms in EDA

| **Evaluation Criterion** | **Tool-Virtualization Middleware (e.g., OneWare)** | **Full-IDE Workspace Virtualization (e.g., DevContainers)** | **Functional Package Managers (e.g., Nix/Guix)** | **Static Package Managers & Extensions (e.g., TerosHDL)** |
|---|---|---|---|---|
| **Virtualization Boundary** | Process-level (Container engine isolating only the per-invocation tool process, bypassing IDE virtualization). | OS-level or Hypervisor-level (Container/MicroVM isolating entire IDE language server and workspace filesystem). | Filesystem-level (Cryptographic hash-based store paths; absolutely no kernel namespace isolation). | None (Native host execution via platform-dependent wrapper shell scripts). |
| **Host Resource Footprint** | Low Disk/RAM (Shares host OS, minimal runtime context), Low CPU overhead (native execution, or binary-translated when the image architecture differs from the host). | High Disk/RAM (Requires full OS runtime per workspace, IDE server backend orchestration), Medium CPU. | High Disk (Immutable deduplicated store retaining historical artifacts), Low RAM/CPU (Native host execution). | Medium Disk (Duplicated binaries per project environment), Low RAM/CPU. |
| **File System I/O Latency** | Native or near-native bind mounts (Extremely low latency). | High/Medium (VirtioFS or 9p protocol overhead mapping host paths to guest OS page caches). | Native host path access (Zero virtualization mapping latency). | Native host path access (Zero virtualization mapping latency). |
| **Security Isolation Boundary** | Capability drop (`--cap-drop ALL`) and `no-new-privileges`; the container runtime supplies the namespace boundary (user namespaces when the daemon runs rootless) under Docker's default seccomp profile. | Rootless namespaces or MicroVM hypervisor physical boundaries (e.g., Virtio-blk/VirtioFS isolation). | Unprivileged native space (Relies entirely on standard POSIX user permissions). | Unprivileged native space (Unrestricted read/write access to host user filesystem). |
| **Execution Transparency** | High (Real-time stdout/stderr multiplexed over the Docker daemon control socket, or via native wrapper scripts). | Medium (Port-forwarded IDE server, highly obfuscated background compilation and LSP translation). | Very High (Direct shell execution via blocking script wrappers). | Very High (Direct binary execution via native command line interface). |

Qualitative comparison; entries reflect the architectural characteristics of each approach.

## 3.4 The Research Gap

The analysis reveals a gap for a solution that is:

1. **Native-First:** The IDE remains on the host (preserving UI responsiveness and OS integration).
2. **Tool-Virtualization Only:** Only the compiler runs in the container (Lightweight).
3. **Transparent:** The container orchestration is hidden behind the IDE's native interaction model, adding only a small, bounded per-invocation cost rather than a workflow-visible one.
4. **Hybrid:** The ability to switch between Native and Container execution at runtime for performance debugging.

This thesis proposes the **Reference Architecture** to fill this specific gap.

# Chapter 4

# Concept and Architecture

This chapter details the conceptual model and architectural design of the OneWare Container Extension. The system is designed to provide transparent containerization of heterogeneous EDA toolchains while maintaining local execution fallbacks, high-performance structured diagnostics, and strict security boundaries.

## 4.1 The Hybrid Strategy Pattern

To address the challenge of runtime execution flexibility (RQ1), the extension implements a modified version of the Gang-of-Four Strategy Pattern [30], termed the *Hybrid Strategy Pattern*.

In standard IDE plugin architectures, a tool execution strategy is statically bound to a specific tool. Under the Hybrid Strategy Pattern, the execution layer dynamically selects the optimal execution provider at runtime based on environmental conditions and user-configured preferences. Figure 4.1 shows the resulting structure, in which a single dispatcher realizes the strategy interface and internally routes each invocation to either the container path or a native-fallback path.

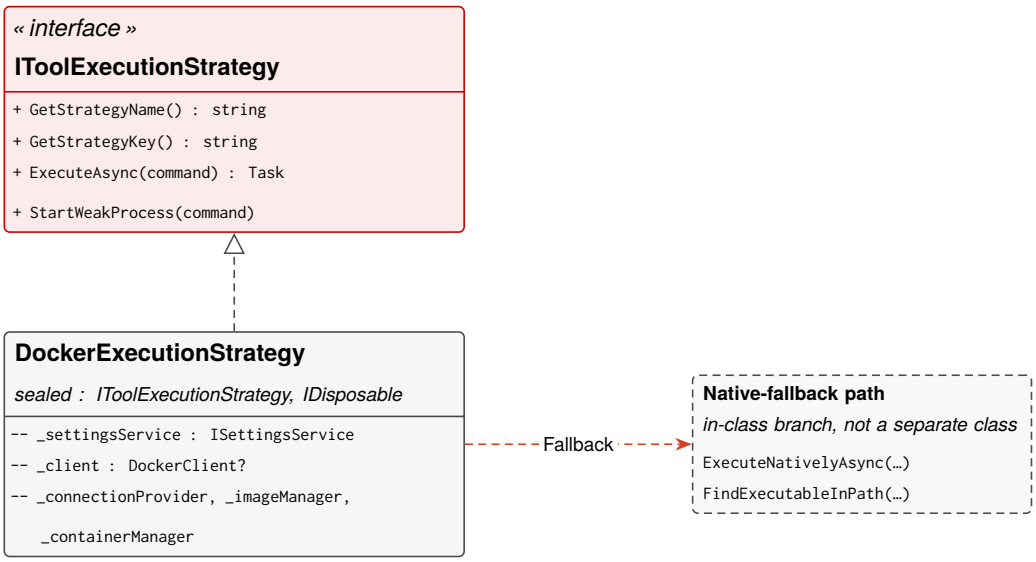



Figure 4.1: Tool-execution strategy structure. `IToolExecutionStrategy` is realized by the single dispatcher `DockerExecutionStrategy`, which routes each invocation either to the container path or to an in-class native-fallback path (the dependency edge) when the container runtime is unavailable and fallback is enabled.

The `DockerExecutionStrategy` implements the `IToolExecutionStrategy` interface and is registered as a selectable execution strategy for each tool. Container routing is opt-in per tool through settings; when a tool is opted in, OneWare dispatches that tool's invocations to this strategy, while native execution remains the default. The decision matrix for execution routing within the strategy is formulated mathematically below.

Let $S_{\text{dock}} \in \{0, 1\}$ represent the connectivity status of the Docker daemon socket, where 1 denotes a responsive, writable socket and 0 represents an unreachable or offline daemon. Let $C_{\text{fallback}} \in \{0, 1\}$ represent the user configuration preference for allowing native host execution fallback (`ContainerExtension_AllowNativeFallback`). Let $P_{\text{native}} \in \{0, 1\}$ denote the presence of the required tool binary within the host system's executable path environment variable (`PATH`).

The chosen execution strategy $E$ is determined by the routing mapping function $f$, defined as:

$$E = f(S_{\text{dock}}, C_{\text{fallback}}, P_{\text{native}}) = \begin{cases} E_{\text{docker}} & \text{if } S_{\text{dock}} = 1 \\ E_{\text{native}} & \text{if } S_{\text{dock}} = 0 \land C_{\text{fallback}} = 1 \land P_{\text{native}} = 1 \\ \text{Err}_{\text{offline}} & \text{if } S_{\text{dock}} = 0 \land C_{\text{fallback}} = 0 \\ \text{Err}_{\text{missing}} & \text{if } S_{\text{dock}} = 0 \land C_{\text{fallback}} = 1 \land P_{\text{native}} = 0 \end{cases} \tag{4.1}$$

where $E_{\text{docker}}$ represents the container execution strategy, $E_{\text{native}}$ denotes the native host execution fallback, and $\text{Err}_{\text{offline}}$ and $\text{Err}_{\text{missing}}$ represent the respective error states.

By implementing this decision matrix, the strategy avoids silent build failures. If the container

engine is offline, it either fails fast with a descriptive configuration diagnostic or transparently falls back to local execution.

## 4.2 Cross-Platform Transport and Namespace Mapping (RQ1)

Ensuring seamless operation across Windows, macOS, and Linux requires resolving operating-system-level differences in transport protocols and file system namespaces.

### 4.2.1 Dynamic Transport Resolution

The Docker Engine API communicates over different local IPC mechanisms depending on the host operating system. The concept introduces a transport resolution sequence, summarized in Figure 4.2:

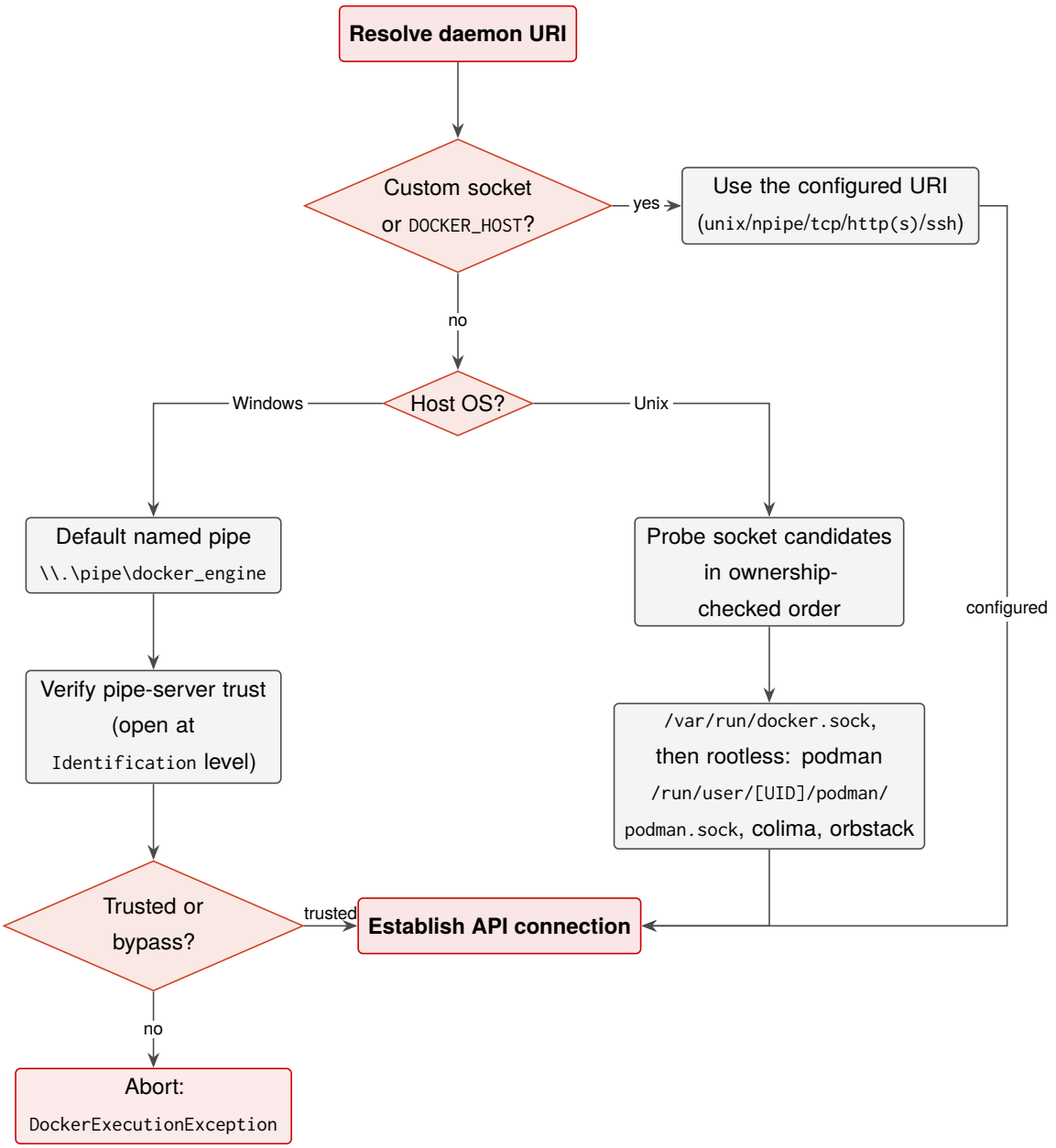



Figure 4.2: Dynamic Transport Protocol Resolution Flowchart

The connection layer automatically checks:

- **Windows Hosts:** Named Pipe transport (`\\.\pipe\docker_engine`) which must be restricted to `TokenImpersonationLevel.Identification` to block privilege escalation.
- **Unix Hosts (macOS/Linux):** Standard Unix Domain Socket (`/var/run/docker.sock`) or rootless alternatives like Podman sockets (typically located at `/run/user/[UID]/podman/podman.sock`).

### 4.2.2 Rootless File Permissions and User-ID Injection

A major architectural issue with containerized execution is file ownership. On Linux hosts, the standard Docker daemon runs as the root user. Consequently, files compiled inside the container (such as synthesis netlists or log outputs) are written back to the host filesystem with root ownership, blocking the local developer from modifying or deleting them without administrative privileges.

To guarantee zero-privilege execution and correct file ownership, the architecture implements dynamic host User ID (UID) and Group ID (GID) injection for rootful container runs [31]. Operating-system-level virtualization relies on user namespaces to isolate user and group ID mappings [32]. A user namespace is instantiated by calling the `clone` system call with the `CLONE_NEWUSER` flag. Upon creation, the process inside the new user namespace is granted a full set of capabilities (such as `CAP_SYS_ADMIN`, `CAP_NET_ADMIN`, and `CAP_SYS_CHROOT`) but only within its namespace boundary. To map these namespace-internal user accounts back to host accounts, the parent process configures the map files at `/proc/[PID]/uid_map` and `/proc/[PID]/gid_map`.

However, writing to these map files from an unprivileged process is restricted. To allow unprivileged users to map multiple host UIDs, the system utilizes SetUID helper binaries: `newuidmap` and `newgidmap`. These utilities query the configuration files `/etc/subuid` and `/etc/subgid`. A typical entry in `/etc/subuid` is formatted as:

```
developer:100000:65536
```

This specifies that the user `developer` is allocated a block of 65536 subordinate UIDs starting at host UID 100000. We can formalize this UID mapping mathematically. Let $\text{UID}_{\text{container}} \in [0, N]$ represent the user ID of a process running inside the container. Let $\text{UID}_{\text{host_user}}$ represent the primary UID of the unprivileged host developer executing the container. Let $\text{UID}_{\text{sub_start}}$ be the subordinate base UID allocated to the user (e.g., 100000). The mapped host UID $\text{UID}_{\text{host_mapped}}$ is defined by the translation function $g : \text{UID}_{\text{container}} \rightarrow \text{UID}_{\text{host_mapped}}$:

$$\text{UID}_{\text{host_mapped}} = \begin{cases} \text{UID}_{\text{host_user}} & \text{if } \text{UID}_{\text{container}} = 0 \\ \text{UID}_{\text{sub_start}} + \text{UID}_{\text{container}} - 1 & \text{if } 0 < \text{UID}_{\text{container}} \leq 65535 \end{cases} \tag{4.2}$$

By establishing this mapping, any files created by the container process running as root (UID 0) are mapped back to the host filesystem with the developer's host UID ($\text{UID}_{\text{host_user}}$), allowing native access. Any files created by other container users (like daemon accounts with UID 1000) are mapped to unprivileged subordinate host UIDs ($\text{UID}_{\text{sub_start}} + 999$), preventing privilege escalation on the host system. In this work the extension does not allocate or compute these subordinate ranges itself: on rootful runtimes it injects the host UID:GID directly through

the container `User` field, and on rootless runtimes it omits `--user` and relies on the runtime's own user-namespace remap. The mapping above therefore describes the underlying runtime mechanism rather than logic implemented by the extension.

When a rootless container is run, the user namespaces map UID 0 (root inside the container) to the unprivileged UID of the user on the host. For standard root-based Docker daemons, this is solved by reading the host developer's UID and GID and injecting them into the container creation payload (`User: "UID:GID"`). The host mapping rules rely on:

- **Subuid/Subgid Mapping:** The files `/etc/subuid` and `/etc/subgid` define ranges of subordinate UIDs/GIDs allocated to users, allowing the rootless runtime to map container-internal users to unprivileged host-side ranges.
- **Bind Mount Mapping:** Dynamic UID/GID injection ensures that files written to mounted host workspaces are mapped back with matching owner permissions, eliminating permission conflicts.

## 4.3 State Machine for Container Lifecycle (Lazy Loading)

To optimize execution latency and resource utilization, the container lifecycle is managed by an event-driven "Lazy Loading" state machine. Rather than pre-allocating containers at IDE startup, containers are managed on demand. Figure 4.3 depicts the resulting eight-stage state machine.

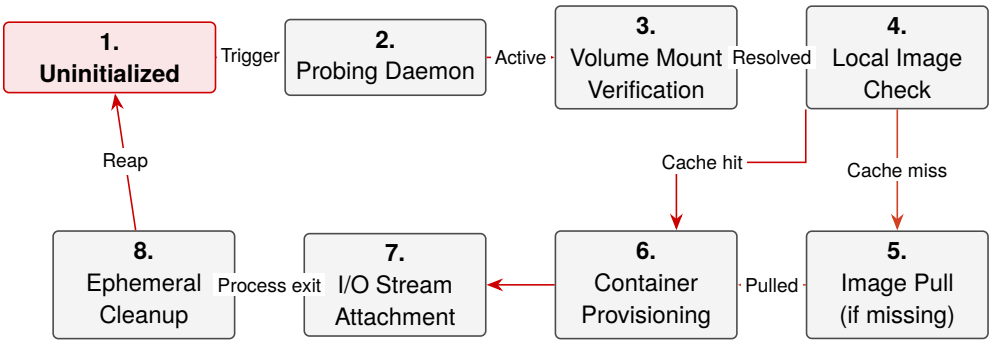



Figure 4.3: State Machine of the 8-stage Lazy-Loading Container Lifecycle

The 8 states of the lifecycle are defined as follows:

1. **Uninitialized:** The tool strategy is idle. No network, RAM, or CPU resources are consumed.
2. **Probing Daemon:** Upon execution trigger, the Docker socket is validated to confirm daemon availability.

3. **Volume Mount Verification:** The host workspace path is resolved, canonicalized, and checked for access permissions.
4. **Local Image Check:** The local image cache is queried for the specific toolchain image via a daemon image inspect; no registry is contacted on this path.
5. **Image Pull (if missing):** If the required image variant is not cached locally, a pull command is dispatched to download the image from the registry.
6. **Container Provisioning:** The container is created via the API with resource limits (cgroups) and path mapping applied.
7. **I/O Stream Attachment:** Standard output and error streams are multiplexed and parsed in real time.
8. **Ephemeral Cleanup:** Upon process termination, the container is destroyed immediately (`AutoRemove: true`) to free system resources.

### 4.3.1 Operational Execution Flow with Fallback Pathways

To complement the abstract state transitions defined by the lazy-loading state machine, Figure 4.4 illustrates the low-level procedural flow executed by the middleware during tool invocation. This execution model accounts for daemon availability, cache validation, path authorization, stream attachment, and termination monitoring.

The execution begins with a daemon probe step. If the connection manager determines that the local container service (Docker/Podman daemon) is inactive or unavailable, the strategy evaluator dynamically routes the compile command to the native host environment, provided that fallback is configured and the necessary binaries are present; otherwise it aborts with a descriptive diagnostic.

If the daemon is active, the runtime first enters the path safety phase: it resolves the host workspace directory and executes the recursive path canonicalization algorithm. If a directory traversal attempt or a circular symbolic link is detected, execution is immediately aborted with a safety exception to protect the host system. Only once the paths are validated does the lifecycle manager ensure the toolchain image is present locally, pulling it from the registry on a cache miss. The runtime then invokes the container creation API, applying UID/GID mapping and cgroups memory/CPU limits.

Following creation, the stream multiplexer attaches to the container's standard output and error pipes. The log decoding engine processes and translates the binary-multiplexed stdout/stderr streams into user-facing console text in real time. Upon process termination, the daemon

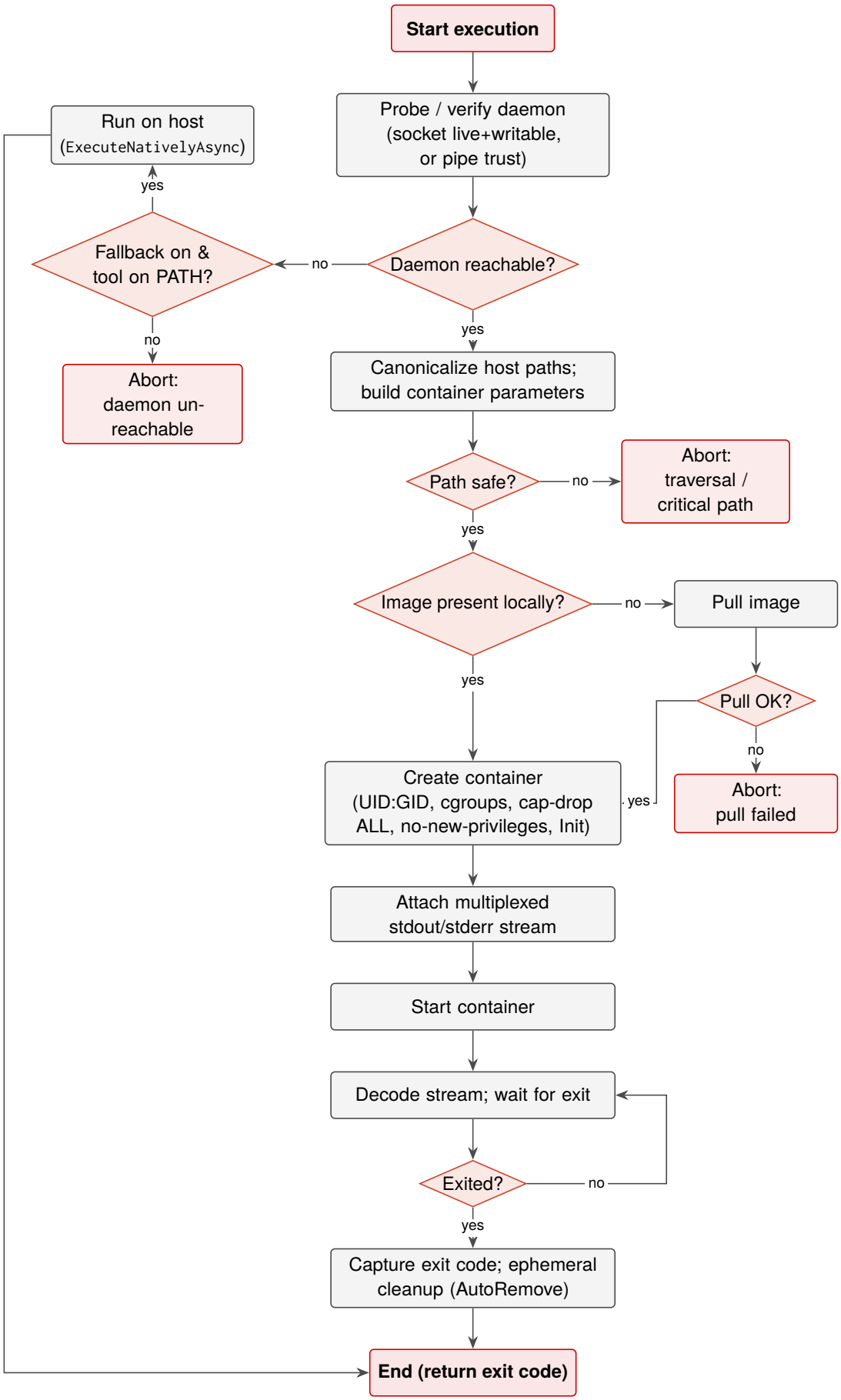



Figure 4.4: Flowchart of Ephemeral Container Execution Lifecycle

sends an exit code, triggering the ephemeral cleanup phase. During cleanup, the container is destroyed, virtual sockets are closed, and allocated resources are returned to the host OS.

To detail the operational transitions and data structures across this 8-stage container lifecycle, Table 4.1 provides a comprehensive map of the triggers, actions, target system interfaces, and success criteria for each state.

Table 4.1: Detailed State Transition Table for the 8-Stage Container Lifecycle

| Stage | State Name | Core Trigger & Actions | API / Syscall Target | Post-Condition |
|---|---|---|---|---|
| 1 | Uninitialized | Tool invocation trigger; checks extension configuration settings | Local configuration query | Environment variables resolved |
| 2 | Probing Daemon | Validate Docker daemon state | GET `/info`, GET `/_ping` | Connection context established |
| 3 | Volume Mount | Resolve, canonicalize, and verify access rights of host project path | `GetCanonicalPath()` | Host mount path securely resolved |
| 4 | Local Image Check | Inspect the local image cache for the toolchain image (no registry contacted) | GET `/images/[name]/json` | Local image presence determined (hit or miss) |
| 5 | Image Pull | Cache-miss transition; download image layer blobs | POST `/images/create` | Image layers downloaded and extracted in cache |
| 6 | Provisioning | Create container with resource constraints and path binds configured | POST `/containers/create` | Ephemeral container created in kernel space |
| 7 | I/O Attachment | Attach and multiplex container standard streams (out, err) | POST `/containers/[id]/attach` | Container-to-host stream piping active |
| 8 | Cleanup | Container execution exits; remove container and free host resources | DELETE `/containers/[id]` (with `force=true`) | Subprocesses reaped; socket connection closed |

Endpoints denote the Docker REST operations invoked at each lifecycle stage.

## 4.4 Security Design Properties

Because the container strategy mounts local workspace directories into the container to perform synthesis, the architecture applies a set of least-privilege defaults. These are stated here as *design properties*; a systematic threat-model analysis of them is the subject of separate work and is deliberately out of scope for this thesis.

- **Non-root execution.** On rootful runtimes the container is run as the host user (`--user`

host UID:GID), so build artifacts are owned by the developer rather than `root`; on rootless runtimes the runtime's own user-namespace remap applies and `--user` is omitted.

- **Reduced kernel surface.** At container creation the extension drops all Linux capabilities, sets `no-new-privileges`, and caps the process count; it relies on the runtime's default seccomp profile rather than programming its own.
- **Workspace path containment.** Bind-mount paths are canonicalized segment-by-segment before reaching the daemon: symbolic links are resolved with cycle detection and a bounded recursion depth, and a path that resolves outside the active workspace root is redirected to an inert in-container target rather than mounted. A residual time-of-check/time-of-use window remains, which a kernel-level resolution primitive (e.g., `openat2` with `RESOLVE_BENEATH`) would close; this is noted as a limitation.
- **Shell-less argument execution.** Commands run as an argument vector (`[executable, args…]`, `UseShellExecute=false`, `tini` as PID 1) with no shell, so shell metacharacters are inert data; a secondary guard additionally rejects arguments that embed shell control operators.
- **Windows transport hardening.** The named-pipe client requests `TokenImpersonationLevel.Identification` rather than the default impersonation level, so a daemon endpoint can authenticate the connecting client without being able to act on its behalf.
- **Resource bounds.** CPU, memory, and process-count limits are applied through the runtime's cgroup controls so that a runaway synthesis or place-and-route run cannot exhaust host resources or starve the IDE; an out-of-memory event is confined to the container's cgroup and surfaces to the IDE as a non-zero exit code (137).

# Chapter 5

# Implementation

The implementation converts the theoretical "Hybrid Strategy Pattern" into a concrete software artifact. This chapter details the software design decisions, the core middleware implementation in C#/.NET 10, and the specific algorithms developed for path validation and log-stream parsing.

## 5.1 Design Decisions

This section outlines the key architectural and platform design choices made during the development of the container extension middleware, focusing on the selected technology stack and transport protocols.

### 5.1.1 Choice of Technology Stack (.NET 10)

The middleware is built on .NET 10 to ensure native interoperability with the OneWare Studio core.

- **Task-based Asynchronous Pattern (TAP):** The async/await pattern is crucial for managing long-running FPGA synthesis jobs (which can take minutes or hours) without freezing the user interface thread. Long-running compilation tasks are run using `Task.Run` with `ConfigureAwait(false)` to prevent context-switching back to the UI thread, ensuring high responsiveness during intensive builds.
- **Live Settings Access:** Operational settings are read live from the host's `ISettingsService` on each execution via `SafeGetSetting`, with change notifications consumed reactively through `GetSettingObservable` (detailed under OneWare Integration below). These settings therefore take effect mid-session without an IDE restart, and their reads require no

extension-level locking. Where the strategy does maintain its own state, it synchronizes explicitly: the validated container-runtime path is memoized behind a `ReaderWriterLockSlim` after first validation and is not re-read mid-session, and weak-process creation is serialized by a `System.Threading.Lock` (Listing 5.5).

- **Cross-Platform BCL:** The `System.Runtime` libraries provide unified APIs for process management and file system I/O across Windows, Linux, and macOS, minimizing platform-specific `#ifdef` blocks and preserving a single codebase.

### 5.1.2 Why Direct Docker Socket vs WSL2?

An alternative approach considered was forcing Windows users to use WSL2 (Windows Subsystem for Linux). However, this was rejected because:

1. **UX Friction:** Setting up WSL2 requires BIOS configuration changes (enabling virtualization features) and local Administrator privileges, which is often blocked in corporate or educational environments.
2. **API Control:** Interacting with `wsl.exe` via command-line parsing is brittle and prone to stdout/stderr encoding bugs. The chosen approach uses the `Docker.DotNet` library to talk directly to the Docker Engine API (using Windows Named Pipes on `\\.\pipe\docker_engine` or Unix sockets on macOS/Linux), providing typed contracts and structured error handling.

## 5.2 Core Middleware (DockerExecutionStrategy)

The `DockerExecutionStrategy` class encapsulates the lifecycle of ephemeral containers. It implements `IToolExecutionStrategy` and combines the `IDisposable` pattern with per-run automatic removal so that containers are reaped—through the Docker Engine API—on disposal, cancellation, or process exit, preventing leaks under the failure modes the managed runtime can observe (a hard host crash is the one case this cannot cover).

### 5.2.1 Architecture Selection (Rosetta 2 Platform Mapping)

Apple Silicon support is handled by a user-selectable image platform rather than automatic detection. The extension exposes a `Platform` setting (a combo box defaulting to `auto`, with `linux/amd64`, `linux/arm64`, and `linux/arm/v7` as explicit choices). When the setting is left at `auto` the pull request leaves the platform unset and the daemon selects a variant; when the user pins a value it is forwarded to the image pull.

Listing 5.1: Settings-driven platform resolution

```
var platform = _settingsService
    .SafeGetSetting<string>(ContainerExtensionModule.PlatformSetting, "auto")?.Trim();
if (!string.IsNullOrEmpty(platform) &&
    !platform.Equals("auto", StringComparison.OrdinalIgnoreCase))
{
    pullParams.Platform = platform; // e.g. "linux/amd64"
}
```

Pinning `linux/amd64` on an Apple Silicon host causes the runtime to pull the x86_64 image and execute it through the Rosetta 2 translation layer; this is the configuration used for the GHDL and Yosys toolchains, whose upstream images are published `linux/amd64` only. If a pinned-platform pull fails, the extension falls back to the host architecture rather than aborting.

## 5.3 Low-Level Named Pipe Impersonation Override

When communicating with the Docker engine on Windows, connections use named pipes. By default, these streams request `TokenImpersonationLevel.Impersonation`. This allows the server to impersonate the client, creating a local privilege escalation (LPE) vector if OneWare Studio runs with elevated privileges.

To mitigate this, the `SecureNamedPipeCredentials` class replaces the `Docker.DotNet` stream factory. It uses reflection to target the private `_streamOpener` field of `ManagedHandler` (within the `Microsoft.Net.Http.Client` namespace) and forces `TokenImpersonationLevel.Identification`. This allows identity verification while blocking the server from acting as the client.

Listing 5.2: Windows Named Pipe Impersonation Override

```
private sealed class SecureNamedPipeCredentials : Docker.DotNet.Credentials
{
    private readonly Uri _endpoint;
    public SecureNamedPipeCredentials(Uri endpoint) { _endpoint = endpoint; }
    public override bool IsTlsCredentials() => false;

    public override System.Net.Http.HttpMessageHandler
     GetHandler(System.Net.Http.HttpMessageHandler innerHandler)
    {
        if (string.Equals(innerHandler.GetType().FullName,
     "Microsoft.Net.Http.Client.ManagedHandler", StringComparison.Ordinal))
        {
            var field = innerHandler.GetType().GetField("_streamOpener",
                System.Reflection.BindingFlags.NonPublic |
                System.Reflection.BindingFlags.Instance);
            if (field != null)
```

```
15              {
16                  var delegateType = field.FieldType;
17                  var method = typeof(SecureNamedPipeCredentials).GetMethod(
18                      nameof(SecureStreamOpenerAsync),
19                      System.Reflection.BindingFlags.NonPublic |
20                      System.Reflection.BindingFlags.Instance);
21                  if (method != null)
22                  {
23                      var d = System.Delegate.CreateDelegate(delegateType, this, method);
24                      field.SetValue(innerHandler, d);
25                  }
26              }
27          }
28          return innerHandler;
29      }
30
31      private async Task<Stream> SecureStreamOpenerAsync(string host, int port, CancellationToken
         token)
32      {
33          var pipeName = _endpoint.LocalPath;
34          var serverName = ".";
35          // Parse serverName and pipeName from URI
36          var pipe = new NamedPipeClientStream(
37              serverName,
38              pipeName,
39              PipeDirection.InOut,
40              PipeOptions.Asynchronous,
41              TokenImpersonationLevel.Identification);
42          await pipe.ConnectAsync(token).ConfigureAwait(false);
43          return pipe;
44      }
45  }
```

We provide a line-by-line walk-through of the `SecureNamedPipeCredentials` implementation in Listing 5.2:

- **Lines 1–4**: Declares the private sealed class `SecureNamedPipeCredentials` inheriting from the library-defined `Docker.DotNet.Credentials` class. It stores the target connection endpoint URI in the `_endpoint` field.
- **Lines 7–10**: The `GetHandler` method hooks the client HTTP handler pipeline. It checks if the inner handler corresponds to the private class `ManagedHandler` inside the `Microsoft.Net.Http.Client` namespace.
- **Lines 11–13**: Uses .NET Reflection to retrieve the private, non-public field `_streamOpener` of type `Delegate` (which is responsible for establishing the underlying OS transport

stream).

- **Lines 14–26**: Obtains the method information of the custom opener method `SecureStreamOpenerAsync` and creates a delegate instance matching the signature of `_streamOpener`. It then overwrites the private delegate field on the HTTP handler using `SetValue`, injecting our custom socket/pipe constructor dynamically.
- **Lines 31–33**: The custom `SecureStreamOpenerAsync` method is invoked whenever the HTTP client needs to send an API request. It parses the local pipe name from the endpoint URI (typically `/pipe/docker_engine`).
- **Lines 34–43**: Configures and instantiates the client stream. Specifically, lines 36–41 construct the BCL `NamedPipeClientStream` using `TokenImpersonationLevel.Identification` to override the default insecure settings. The stream is connected asynchronously on line 42 and returned on line 43.

### 5.3.1 Why Reflection Is Required

The downgrade cannot be expressed through the public API. `Docker.DotNet`'s `ManagedHandler` holds the stream-opening delegate (`_streamOpener`) in a private field, so the wrapper uses reflection—`GetField` with `BindingFlags.NonPublic | BindingFlags.Instance`, then `SetValue` to install a delegate bound to `SecureStreamOpenerAsync`—to replace the default opener with one that constructs the pipe at `TokenImpersonationLevel.Identification`. At this level a pipe server may verify the connecting client's identity but cannot impersonate its token to act on its behalf, so even a rogue endpoint that calls `ImpersonateNamedPipeClient()` obtains no usable privilege. The detailed kernel-level mechanics of named-pipe impersonation and the security-reference-monitor checks that enforce this are standard Windows behavior; their full treatment, and a threat-model analysis of the transport, are out of scope here and are noted as design properties in Section 4.4. Figure 5.1 shows the resulting connection sequence.

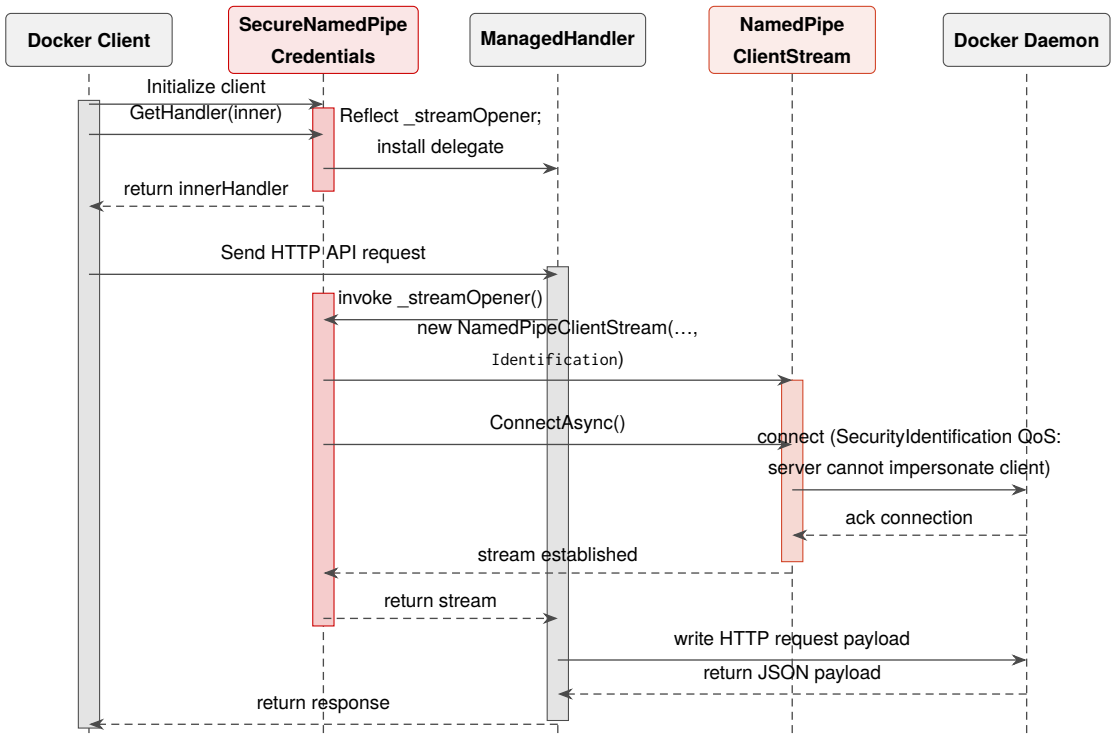



Figure 5.1: Sequence of the reflection-based named-pipe stream-opener replacement

## 5.4 Log Stream Demultiplexing and Parsing

The raw output from the container is a multiplexed stream of `stdout` and `stderr`. The implementation parses this stream in real time.

### 5.4.1 Stream Multiplexing

Unlike a standard operating system pipe, the Docker API wraps all output multiplexing in an 8-byte frame header, as shown in Figure 5.2.

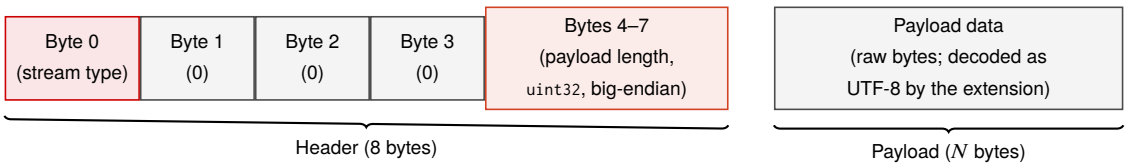



Figure 5.2: Docker API stream multiplexing: 8-byte frame header and payload

The extension consumes Docker.DotNet's `MultiplexedStream`, whose `ReadOutputAsync` demultiplexes this 8-byte-framed attach stream and tags each chunk with its target. Each frame carries:

- `Byte 0`: Stream identifier (0 = stdin, 1 = stdout, 2 = stderr).

- `Bytes 1-3`: Reserved bytes (must be 0).
- `Bytes 4-7`: Payload length encoded as a 32-bit big-endian integer.

The extension routes each demultiplexed chunk by its target identifier (stdout or stderr), converts the payload bytes to UTF-8 plaintext, and streams them line-by-line to the diagnostic log handlers.

### 5.4.2 Regex Source Generators

The extension uses .NET's source-generated regular expressions on hot paths—input validation and the scrubbing of secrets from telemetry before logs are written—so that the matcher is generated at compile time rather than built at runtime. For example, the image-reference validator is declared as:

Listing 5.3: Source-generated image-reference validation

```
[GeneratedRegex(
    @"^[a-zA-Z0-9][a-zA-Z0-9._\-]*(:\d{1,5})?(/[a-zA-Z0-9._\-]+)*(:[a-zA-Z0-9._\-]+)?(@sha256:[a-fA-F0-9]{64})?$",
    RegexOptions.ExplicitCapture | RegexOptions.IgnoreCase | RegexOptions.NonBacktracking,
    matchTimeoutMilliseconds: 1000)]
private static partial Regex ImagePatternRegex();
```

By using the `[GeneratedRegex]` attribute, the C# compiler emits specialized matcher code in Intermediate Language (IL) during the build. This eliminates the runtime compilation overhead and Large Object Heap (LOH) allocations associated with `new Regex()`, and the `NonBacktracking` option bounds worst-case matching time. The same technique backs the telemetry secret-scrubbing patterns (API keys, tokens, private keys, and credential-bearing URIs). Mapping raw tool output to log levels in the IDE is performed by lightweight line parsing rather than a source-generated diagnostic grammar.

## 5.5 Recursive Segment-by-Segment Canonicalization

To block directory traversal attacks (where symbolic links are used to escape the mounted project directory), host workspace paths are recursively canonicalized segment-by-segment prior to container creation.

The `GetCanonicalPath` method maps path components, resolves symlinks, and maintains a recursion count to catch circular links.

Listing 5.4: Path Canonicalization implementation

```
private static string GetCanonicalPath(string path)
{
```

```
3       if (string.IsNullOrWhiteSpace(path))
4       {
5           throw new ArgumentException("Path cannot be null or empty.", nameof(path));
6       }
7
8       var seenSymlinks = new HashSet<string>(OperatingSystem.IsWindows() ?
         StringComparer.OrdinalIgnoreCase : StringComparer.Ordinal);
9
10      string ResolveCanonicalInternal(string currentPath, int depth)
11      {
12          if (depth > 40)
13          {
14              throw new ContainerExtension.DockerExecutionException("Too many levels of symbolic
         links.");
15          }
16
17          string absolutePath = currentPath;
18          if (!Path.IsPathRooted(absolutePath))
19          {
20              absolutePath = Path.Combine(Directory.GetCurrentDirectory(), absolutePath);
21          }
22
23          string root = Path.GetPathRoot(absolutePath) ?? (OperatingSystem.IsWindows() ? "C:\\" :
        "/");
24          if (string.IsNullOrEmpty(root))
25          {
26              root = OperatingSystem.IsWindows() ? "C:\\" : "/";
27          }
28
29          string remainder = absolutePath.Substring(root.Length);
30          var separatorChars = new char[] { '/', '\\' };
31          var components = remainder.Split(separatorChars, StringSplitOptions.RemoveEmptyEntries);
32
33          string current = root;
34
35          foreach (var component in components)
36          {
37              if (string.Equals(component, ".", StringComparison.Ordinal))
38              {
39                  continue;
40              }
41
42              if (string.Equals(component, "..", StringComparison.Ordinal))
43              {
44                  var parent = Path.GetDirectoryName(current);
45                  current = parent ?? root;
46                  continue;
```

```
47              }
48
49              string next = Path.Combine(current, component);
50              bool isSymlink = false;
51              string? target = null;
52
53              try
54              {
55                  var info = new DirectoryInfo(next);
56                  if (info.Exists && info.Attributes.HasFlag(FileAttributes.ReparsePoint))
57                  {
58                      target = info.LinkTarget;
59                      isSymlink = !string.IsNullOrEmpty(target);
60                  }
61              }
62              catch (Exception ex) when (ex is not OutOfMemoryException) { }
63
64              if (isSymlink && target != null)
65              {
66                  string canonicalSymlink = Path.GetFullPath(next);
67                  if (!seenSymlinks.Add(canonicalSymlink))
68                  {
69                      throw new ContainerExtension.DockerExecutionException($"Circular symbolic
    link detected: '{next}'");
70                  }
71
72                  try
73                  {
74                      string resolvedTarget = Path.IsPathRooted(target)
75                          ? ResolveCanonicalInternal(target, depth + 1)
76                          : ResolveCanonicalInternal(Path.Combine(current, target), depth + 1);
77                      current = resolvedTarget;
78                  }
79                  finally
80                  {
81                      seenSymlinks.Remove(canonicalSymlink);
82                  }
83              }
84              else
85              {
86                  current = next;
87              }
88          }
89          return Path.GetFullPath(current);
90      }
91      return ResolveCanonicalInternal(path, 0);
92  }
```

A detailed line-by-line analysis of the `GetCanonicalPath` method in Listing 5.4 reveals its security mechanics:

- **Lines 3–6**: Performs standard pre-condition validation, throwing an exception if the input path string is empty or contains only whitespace characters.
- **Line 8**: Instantiates a hash set `seenSymlinks` to record the resolved path of all symlinks encountered during recursion. This is essential to prevent infinite loops from circular links. Windows paths are handled case-insensitively, while Unix paths are case-sensitive.
- **Lines 10–15**: Defines the local helper function `ResolveCanonicalInternal` with a recursion limit. If the depth exceeds 40, the method aborts, bounding the symlink-chain length against long-chain denial-of-service attempts. Genuine circular links are caught separately by the `seenSymlinks` set (Lines 64–82).
- **Lines 17–21**: Root-resolves the path if it is not already absolute, combining it with the current working directory.
- **Lines 23–31**: Extracts the root directory and splits the path remainder into individual name segments using directory separators.
- **Lines 35–47**: Loops through each segment. Skip patterns are applied for single dot `.` segments, and parent references `..` are resolved by traversing back up to the parent directory.
- **Lines 49–60**: Builds the path of the next child component. It uses a `DirectoryInfo` object to check if the component exists on the disk and represents a reparse point (symlink). If it is a symlink, it reads the target path.
- **Lines 64–82**: If a symlink is found, its target is recursively canonicalized by calling `ResolveCanonicalInternal` with an incremented depth. The symlink path is added to `seenSymlinks` beforehand and removed in a `finally` block to ensure accurate tracking during traversal.
- **Lines 84–87**: If the component is a regular directory, it simply appends it. The final resolved path is returned as a fully qualified path.

## 5.6 Weak Process Lifecycle & Cancellation Propagation

The OneWare Studio plugin interface expects standard operating system processes (`Process`) returned via the `IToolExecutionStrategy` interface to monitor execution status and propagate

user cancellation. Since container execution is carried out asynchronously via HTTP requests to the Docker socket rather than local child processes, a wrapper strategy was required.

The extension implements `StartWeakProcess`, which starts a dummy process (`ping` on Windows, `sleep` on Unix) to satisfy the IDE's process handle contracts, and hooks its lifecycle events to the container's execution.

Listing 5.5: Weak Process Lifecycle Wrapper

```
public WeakReference<Process> StartWeakProcess(ToolCommand command)
{
    lock (WeakProcessLock)
    {
        var runCts = new CancellationTokenSource();
        var dummyProcess = new Process();
        dummyProcess.StartInfo = new ProcessStartInfo
        {
            FileName = OperatingSystem.IsWindows() ? "ping" : "sleep",
            Arguments = OperatingSystem.IsWindows() ? "127.0.0.1 -n 86400" : "86400",
            CreateNoWindow = true,
            UseShellExecute = false
        };
        dummyProcess.EnableRaisingEvents = true;
        dummyProcess.Exited += (s, e) =>
        {
            try { runCts.Cancel(); }
            catch (ObjectDisposedException) { }
        };

        try { dummyProcess.Start(); }
        catch (Exception ex)
        {
            ContainerTelemetry.TrackError("DockerExecutionStrategy", "Failed to start dummy
    process in StartWeakProcess", ex);
            dummyProcess = new Process();
        }

        _ = Task.Run(async () =>
        {
            try
            {
                await ExecuteAsync(command, runCts.Token).ConfigureAwait(false);
            }
            catch (Exception ex)
            {
                ContainerTelemetry.TrackError("DockerExecutionStrategy", "StartWeakProcess task
    crashed", ex, command.Executable);
            }
```

```
38              finally
39              {
40                  try
41                  {
42                      if (!dummyProcess.HasExited) { dummyProcess.Kill(true); }
43                  }
44                  catch { }
45                  try { runCts.Dispose(); }
46                  catch { }
47              }
48          }, CancellationToken.None);
49
50          return new WeakReference<Process>(dummyProcess);
51      }
52  }
```

We trace the operational flow of `StartWeakProcess` in Listing 5.5:

- **Lines 3–6**: Spawns a critical synchronization lock `WeakProcessLock` to ensure thread-safe process handle creation. It then creates a cancellation token source.
- **Lines 7–13**: Instantiates a dummy `Process` using the native operating system shell executables. On Windows, it runs `ping 127.0.0.1 -n 86400` to idle, while on Unix it runs `sleep 86400`.
- **Lines 14–19**: Hooks the process exit event. If the process is terminated (e.g., killed by the IDE or manually stopped by the user), the event handler immediately cancels the token source.
- **Lines 21–26**: Starts the dummy process and captures any exceptions.
- **Lines 28–48**: Dispatches a background task using `Task.Run` to execute the container synthesis run. This task runs asynchronously, taking the cancellation token from the dummy process. If the token is canceled, it calls the Docker API to stop the container.
- **Lines 38–47**: The `finally` block ensures that if the container completes its task, the dummy process is cleaned up immediately, freeing the IDE process slot.

If the user clicks “Cancel” inside OneWare Studio, the IDE kills the returned dummy process. The dummy process’s `Exited` event triggers cancellation of the background `CancellationTokenSource`, forcing the container runtime to send a stop signal to the Docker API and tear down the container.

## 5.7 Argument-Vector Construction and Namespace Mapping (DockerCommandBuilder)

Rather than reconstructing a shell command string, the `DockerCommandBuilder` produces an *argument vector* that the container executes directly, with no shell. Each token from the IDE's `ToolCommand` has its host path mapped to the corresponding container path where applicable, and path separators are normalized for the Linux container filesystem. Because the tokens are passed as a vector and never concatenated into a shell line, shell metacharacters cannot be reinterpreted—they occupy inert argument slots, a property the project's `CommandInjectionProofOfConceptTests` suite verifies directly. As defense in depth, a separate check in `DockerExecutionStrategy.ExecuteAsync`—applied before the vector is built—rejects an executable name containing any shell metacharacter and rejects any argument that embeds a command separator or nested-shell construct (`&&`, `||`, command substitution, or—outside Yosys scripts—;).

Listing 5.6 shows the token-mapping and emission loop.

Listing 5.6: Argument-vector construction (shell-less)

```
var cmdTokens = new List<string> { executable };
foreach (var arg in command.Arguments)
{
    // Map a host path to its container-mount equivalent where applicable;
    // otherwise keep the token unchanged.
    var mapped = ShouldMapArgument(arg)
        ? MapPathToContainer(arg, workingDirFull, workingDirCanonical)
        : arg;

    // Each mapped token becomes one argv element. No shell escaping or quoting is
    // applied: the token is delivered verbatim to the program through execve,
    // exactly as OneWare's native execution would, so shell metacharacters are
    // inert data rather than syntax.
    cmdTokens.Add(NormalizeSeparators(mapped));
}
// cmdTokens becomes the container Cmd vector [executable, args...], executed
// without a shell (tini as PID 1, HostConfig.Init = true).
```

A walk-through of the routine in Listing 5.6:

- **Path resolution and mapping:** each raw argument is tested with `ShouldMapArgument`; path-like arguments have their host directory rewritten to the container mount root (`/workspace`) using the canonicalized host project path.
- **Separator normalization:** backslashes are converted to forward slashes (`/`) for the

Linux container filesystem.

- **Verbatim argv emission:** each mapped token is appended as exactly one `Cmd` element—no escaping, quoting, or per-token rejection. Because the vector is handed to `execve` through the container entrypoint (`tini`), a metacharacter such as `&&` lands in a single argument slot and is never parsed as shell syntax.
- **Defense-in-depth guard (elsewhere):** the shell-operator rejection is not part of this builder. It lives in `DockerExecutionStrategy.ExecuteAsync`, where the executable name is screened against a `SearchValues<char>` set of shell metacharacters and a narrow per-argument denylist rejects `&&`, `||`, command substitution, and `;` (the last suppressed for Yosys). Because safety already follows from the shell-less argv model, this check is a secondary boundary, not the primary one.

## 5.8 Dynamic Registry Client and Tag Discovery

To populate the toolchain-version selector with the tags available for an image, the architecture implements a dedicated registry client (`RegistryClient`) that queries public registries over HTTPS, guards against server-side request forgery, and caches results briefly.

Listing 5.7 shows the tag-query entry point and its Docker Hub backend.

Listing 5.7: Registry tag query and caching (simplified)

```
// Public entry point: returns the tags for an image reference, cached briefly.
public static async Task<List<string>> FetchTagsAsync(
    string imageReference, CancellationToken cancellationToken = default)
{
    // Serve from cache while fresh: 60 s for a populated list, 10 s if empty.
    if (TagsCache.TryGetValue(imageReference, out var cached))
    {
        var maxAgeMs = cached.tags.Count == 0 ? 10_000 : 60_000;
        if (Environment.TickCount64 - cached.cacheTimeTicks < maxAgeMs)
            return new List<string>(cached.tags);      // hand out a copy
    }

    var parts = ParseImageReference(imageReference);   // registry/namespace/repo
    // SSRF gate (omitted): reject hosts that resolve to loopback/internal addresses.
    List<string> tags = parts.Registry switch
    {
        "" or "docker.io" or "registry-1.docker.io"
            => await FetchDockerHubTagsAsync(parts.Namespace, parts.Repository,
                                             cancellationToken),
        "ghcr.io" => await FetchGhcrTagsAsync(parts.Namespace, parts.Repository,
                                              cancellationToken),
        _ => await FetchGenericV2TagsAsync(parts.Registry, parts.Namespace,
                                           parts.Repository, cancellationToken),
    };
    AddToCache(imageReference, tags);                  // stamps insertion time
    return tags;
```

```
27  }
28
29  // Docker Hub backend: anonymous GET, no bearer token; source-generated JSON.
30  private static async Task<List<string>> FetchDockerHubTagsAsync(
31      string ns, string repo, CancellationToken ct)
32  {
33      var url = $"https://hub.docker.com/v2/repositories/{ns}/{repo}"
34              + "/tags?page_size=20&ordering=last_updated";
35      using var request = new HttpRequestMessage(HttpMethod.Get, url);
36      request.Headers.Accept.Add(new MediaTypeWithQualityHeaderValue("application/json"));
37      using var response = await SendWithRetryAsync(request, ct);
38      response.EnsureSuccessStatusCode();
39
40      using var stream = await response.Content.ReadAsStreamAsync(ct);
41      var res = await JsonSerializer.DeserializeAsync(
42          stream, RegistryJsonContext.Default.HubResponse, ct);  // reflection-free
43      var list = new List<string>();
44      foreach (var r in res?.Results ?? [])
45          if (!string.IsNullOrEmpty(r.Name)) list.Add(r.Name);
46      return list;
47  }
```

A detailed walk-through of the registry query routine in Listing 5.7:

- **Local Caching:** To avoid Docker Hub's per-IP rate limits on unauthenticated requests, the client checks its in-memory cache before making external network calls. The cache is managed using a thread-safe dictionary with a short absolute-age freshness window—60 seconds for a populated tag list, 10 seconds for an empty result—evaluated since insertion, so an entry is not restamped when it is read.

- **Per-Registry Dispatch and Authentication:** `FetchTagsAsync` parses the reference and routes it to the matching backend. The Docker Hub listing (`hub.docker.com/v2/repositories/.../tags`) is unauthenticated; the GitHub Container Registry (`ghcr.io`) obtains a short-lived pull-scope bearer token from GHCR's well-known token endpoint before listing tags, whereas generic `/v2/` registries obtain one from the token endpoint advertised in the registry's `WWW-Authenticate` challenge after an initial unauthorized response. An SSRF guard rejects any reference whose host resolves to a loopback or internal address.

- **Source-Generated Deserialization:** The JSON tag list is deserialized directly from the response stream through the source-generated `RegistryJsonContext` (a compile-time `JsonSerializerContext`, avoiding reflection for AOT compatibility) into `HubResponse` for Docker Hub or `GhcrTagsResponse` for GHCR. The extracted tag names populate the toolchain-version drop-down menu in OneWare Studio, allowing users to select different toolchain versions.

## 5.9 OneWare Integration

The integration layer bridges the internal `IToolExecutionStrategy` service to the OneWare Studio plugin infrastructure (`OneWare.Essentials`).

- **Dependency Injection:** The `DockerExecutionStrategy` is registered as a Singleton in the `IServiceCollection` within the module's service registration.
- **Live Settings Reads:** Operational settings—image, platform, resource limits, and `ContainerExtension_AllowNativeFallback`—are read live from `ISettingsService` on each execution via `SafeGetSetting`, so changes take effect without an IDE restart. Separately, the module subscribes to the Telemetry Retention setting through `ISettingsService.GetSettingObservable` and purges the telemetry log immediately when it is set to `None`.

# Chapter 6

# Evaluation

This chapter presents the empirical results of the thesis. It validates the architectural claims regarding Cross-Platform Compatibility (RQ1) and Performance Viability (RQ3) through quantitative benchmarking and qualitative failure analysis.

## 6.1 Data Status and Threats to Validity

The measurements reported in this chapter are complete for all three machines: Machine 1 (Apple Silicon), Machine 2 (Linux `x86_64`), and Machine 3 (Windows `x86_64`). Three properties of the data bound its interpretation and are stated here up front.

First, the macOS arm is *emulation-confounded*. The reference toolchain image is published `linux/amd64` only, so on the ARM64 host it executes under the runtime's Rosetta 2 translation layer, whereas the native baseline is ARM64. A macOS native-versus-container comparison therefore measures containerization *and* instruction-set emulation together; any resulting overhead factor would be only an *upper bound* for the emulated case and would not isolate the cost of the integration layer, so no native baseline is captured on this host. Machine 3 removes this confound for the marginal-cost axis: it runs the same `linux/amd64` image natively under WSL2, without Rosetta translation, and its extension-overhead figures (Section 6.4.3) confirm the macOS finding on un-emulated hardware.

Second, a valid native-versus-container factor—the continuous multiplier $\alpha$ of Section 6.5.1—is isolable on *only one* of the three hosts, and that measurement was performed on Machine 2. It requires a host on which the native toolchain and the containerized toolchain are the *same* build. Machine 1 cannot provide it, because its native baseline would be ARM64 while the container is emulated `amd64`. Machine 3 cannot either, for a different reason established empirically here: the native Windows `oss-cad-suite` release is a MinGW/GCC

Windows build, whereas the container carries the Clang/Linux build of the identical source and commit, so a version-parity gate (Section 6.5.2) correctly rejected the Windows native-versus-container comparison. The Linux `x86_64` host (Machine 2), whose native package *is* the same Clang/Linux build shipped in the image, satisfies the gate and yields the $\alpha$ baseline reported in Section 6.4.2.

Third, output determinism is assessed *across* all three platforms. With Machines 1, 2, and 3 present, the per-artifact SHA-256 comparison is a genuine cross-machine, cross-architecture (one ARM64-hosted versus two `x86_64`-hosted executions) test rather than the single-platform, informational check it was before; the result is reported in Section 6.5.3. On the two hosts without a same-build native baseline (Machines 1 and 3) the cleanly attributable timing quantity remains the *marginal* cost the extension adds beyond raw containerization—the strategy-versus-CLI difference—while Machine 2 additionally yields the container-versus-native factor.

## 6.2 Experimental Setup

To ensure the reproducibility of the results, the evaluation was conducted on a standardized hardware testbed representing a typical high-end developer environment.

### 6.2.1 Hardware Testbed Configuration

To validate the performance characteristics of the containerized execution environment across different operating systems and CPU architectures, three separate evaluation machines were configured: one Apple Silicon (ARM64) host and two `x86_64` hosts running Linux and Windows respectively. All three machines have been measured; Machine 2 (Linux `x86_64`) is the native-baseline ($\alpha$) testbed identified in Section 6.1, the only host that admits a same-build native comparison.

**Machine 1 (macOS Host - Apple Silicon):**

- **Device:** Apple MacBook Pro (16-inch, 2024)
- **SoC:** Apple M4 Max (16-Core CPU: 12 Performance Cores, 4 Efficiency Cores; 40-Core GPU)
- **Memory:** 128 GB Unified Memory
- **OS:** macOS Tahoe 26.6 (Darwin 25.6.0)
- **Container Runtime:** OrbStack (Docker Engine v29.4.0) with Rosetta 2 translation layer enabled

- **Toolchain:** .NET SDK 10.0.301, Python 3.9.6

**Machine 2 (Linux Host - x86_64):**

- **CPU:** Intel Core Ultra 9 285H (Arrow Lake; 6 Performance + 8 Efficiency + 2 Low-Power-Efficiency cores, 16 cores / 16 threads)
- **Memory:** 64 GB
- **OS:** Ubuntu 26.04 LTS (kernel 7.0.0-15-generic, `performance` CPU governor)
- **Container Runtime:** Docker Engine v29.1.3 (native Linux containers, `overlayfs` storage driver, `x86_64`; no emulation or WSL layer)
- **Toolchain:** .NET SDK 10.0.301, Python 3.11.6

**Machine 3 (Windows Host - x86_64):**

- **CPU:** Intel Core Ultra 7 155H (Meteor Lake; 6 Performance + 8 Efficiency + 2 Low-Power-Efficiency cores, 16 cores / 22 threads)
- **Memory:** 32 GB
- **OS:** Windows 11 (build 26200)
- **Container Runtime:** Docker Desktop v29.6.1 (Linux containers, WSL2 backend, `x86_64`)
- **Toolchain:** .NET SDK 10.0.301, Python 3.12.10

The host, CPU, memory, and runtime figures for Machines 2 and 3 are those captured programmatically at run time (`machine.json`); on both `x86_64` hosts the container image executes natively with no instruction-set translation.

## 6.3 Methodology

The performance overhead was measured with a purpose-built benchmarking harness, `ContainerBenchmarkHarness` (`src/ContainerBenchmarkHarness/Program.cs`), driven by a Python orchestration layer (`benchmark.py`, `run_evaluation.py`). The harness does not approximate the extension; it constructs a real `DockerExecutionStrategy` and invokes the same `ExecuteAsync` code path that OneWare Studio uses in production, including image resolution, container lifecycle management and stream demultiplexing. Timing, statistics and result serialization are performed in Python, so a single measurement protocol applies uniformly to every execution backend.

### 6.3.1 Harness Build and Timing Model

The harness is a standard .NET Release build executed under the Just-In-Time (JIT) compiler. Its code paths are reflection-free and therefore Native-AOT compatible—the project sets the AOT-analysis suppression properties—but it is not AOT-published in this evaluation; the reported figures are produced by the JIT build. JIT warm-up and any first-run container setup are excluded from the measurement by an unrecorded warm-up phase (below), and per-iteration timing is captured on the Python side with `time.perf_counter()` around each invocation, so the recorded interval is the end-to-end wall-clock cost of the dispatch as observed by the caller. This timer is used identically for all backends, which makes the cross-backend deltas—the quantity of interest—independent of any .NET-internal clock.

### 6.3.2 Environmental Control and Hardware Isolation

To guarantee the validity and reproducibility of the performance metrics, strict environmental controls were applied to the hardware testbeds during benchmarking:

- **Process Isolation:** All user-space background processes (e.g., development environments, communication clients, web browsers) were terminated on the host systems to eliminate CPU scheduling conflicts and context-switching interference.
- **Power State Configuration:** The testbed machine was connected to a dedicated external AC power source, and the operating system power manager was configured to a high-performance profile. This prevents the OS from dynamically scaling CPU clock frequencies or entering low-power states during execution, which would skew the timing.
- **Thermal Stability:** No artificial cool-down was inserted between iterations. The per-iteration container lifecycle (teardown and re-creation between invocations) provides natural spacing, and thermal throttling (where the hardware dynamically decreases the CPU frequency to protect from heat buildup) would surface under the CV-adaptive repetition protocol as elevated variance, which the reported CVs do not show.
- **Asymmetrical Core Scheduling (macOS):** The Apple M4 Max SoC contains an asymmetrical core configuration consisting of 12 Performance (P) cores and 4 Efficiency (E) cores. The macOS thread scheduler manages thread affinity dynamically. To prevent vCPU threads from being scheduled onto the slower E-cores during benchmarking (which would artificially inflate the mean execution time), the system load was verified as idle ($< 1\%$) prior to execution, ensuring the XNU scheduler dispatched all benchmark threads to high-frequency P-cores.

### 6.3.3 Execution Backends

Each workload is evaluated across three execution backends, selected by `benchmark.py --backend`:

1. **Native.** The tool binary is invoked directly on the host as the reference baseline. This backend is optional (`run_evaluation.py --with-native`) and is valid only when the native toolchain is the *same build* as the one inside the image, verified by a version-parity gate before the comparison is admitted (Section 6.5.2). It was not captured on Machine 1 (a macOS native run measures containerization *and* emulation together) and was rejected on Machine 3 (the native Windows `oss-cad-suite` is a MinGW build, the container a Clang build of the same source); it was admitted on the Linux `x86_64` host (Machine 2), whose native package is the same Clang/Linux build as the image, yielding the baseline reported in Section 6.4.2.

2. **CLI Container (`cli`).** The workload is wrapped in `docker run --rm` with the workspace bind-mounted at `/workspace`. This isolates the cost of containerization itself, independent of the extension.

3. **Strategy (`strategy`).** The workload runs through the real `DockerExecutionStrategy.ExecuteAsync` path via `ContainerBenchmarkHarness`—the identical code path shipped in the extension. Telemetry and SDK logging are disabled in the harness so their writes do not run on the measured thread and confound the timing.

The load-bearing comparison on the emulation-confounded macOS host is `strategy` versus `cli`: because both share the identical container and Rosetta translation path, their difference isolates the marginal cost the extension adds beyond raw containerization, and this quantity needs no native baseline.

### 6.3.4 Timing and Statistical Protocol

The benchmarking protocol comprises an unrecorded warm-up phase followed by a statistically controlled measurement phase.

- **Warm-up.** Two unrecorded warm-up runs are executed for each backend, priming OS and CPU caches and, for the containerized backends, the Docker daemon's connection structures and local image layer cache, so the measured iterations reflect steady state rather than first-run setup.

- **CV-adaptive repetition.** Each suite takes a floor of $n = 30$ measured iterations and continues sampling until the coefficient of variation (CV) falls to or below a 15 % target

or a cap of 60 iterations is reached, so noisy workloads are sampled more heavily and quiet ones are not over-sampled.

Each per-iteration interval is captured in Python with `time.perf_counter()`. From the recorded samples the harness computes the arithmetic mean with a two-sided 95 % Student-$t$ confidence interval and the CV. Each container backend is compared against the reference with Welch's unpaired $t$-test for unequal variances, reporting the Welch–Satterthwaite degrees of freedom, a two-sided $p$-value, and Cohen's $d$ effect size; $p$-values are Holm–Bonferroni corrected for the family-wise error rate across the whole workload × platform × backend matrix. In-container peak memory is read from the Docker stats stream (`container_peak_mem_mb`) rather than host-side accounting. The one-time image-acquisition cost is measured separately as a dedicated `lifecycle_floor` workload so it never contaminates the steady-state figures. The harness additionally records the median and the 95th percentile, but the tables in this chapter report dispersion through the CV and confidence interval.

## 6.4 Quantitative Results (Performance)

The overhead axis is characterized across the seven-workload FPGA corpus—iCE40 and ECP5 synthesis, place-and-route and bitstream packing, plus Icarus Verilog simulation—each a single-tool dispatch mirroring how OneWare drives the toolchain. A bare no-op container invocation (`lifecycle_floor`) is measured separately as the fixed per-invocation floor. The axis is reported on Machine 1 (emulated `amd64`, Section 6.4.1), Machine 2 (native `amd64` with a same-build native baseline, Section 6.4.2), and Machine 3 (native `amd64`, Section 6.4.3); all figures are the committed harness output for the `macos-arm64`, `linux-amd64`, and `windows-amd64` platforms respectively.

### 6.4.1 Extension Overhead on Machine 1 (macOS, Apple Silicon)

Table 6.1 reports, for each workload, the mean wall-clock time of the raw Docker CLI backend and of the real strategy backend, with 95 % confidence intervals and coefficients of variation, and the extension overhead factor (strategy mean divided by CLI mean). Because no native baseline is captured on this emulation-confounded host (Section 6.1), the reported ratio isolates the marginal cost of the integration layer over raw containerization rather than the cost of containerization itself. Every strategy-versus-CLI difference is statistically significant after Holm–Bonferroni correction ($p \approx 0$).

Table 6.1: Extension overhead on Machine 1 (Apple M4 Max, macOS Tahoe 26.6, Docker 29.4.0). CLI and strategy backends only; $n = 30$ unless noted; 95 % Student-$t$ CIs. Ext. overhead is the strategy-to-CLI mean ratio.

| | CLI container | | | Strategy (as shipped) | | | |
|---|---|---|---|---|---|---|---|
| **Workload** | **Mean (s)** | **95 % CI** | **CV%** | **Mean (s)** | **95 % CI** | **CV%** | **Ext. ov.** |
| synth_ice40_yosys | 1.3456 | [1.333, 1.358] | 2.5 | 1.5042 | [1.495, 1.513] | 1.6 | 1.12 |
| synth_ecp5_yosys | 1.1647 | [1.154, 1.176] | 2.5 | 1.3111 | [1.296, 1.327] | 3.2 | 1.13 |
| pnr_ice40_nextpnr | 0.6309 | [0.621, 0.641] | 4.3 | 0.8207 | [0.813, 0.829] | 2.6 | 1.30 |
| pnr_ecp5_nextpnr | 1.0036 | [0.993, 1.015] | 3.0 | 1.1866 | [1.178, 1.195] | 2.0 | 1.18 |
| pack_ice40_icepack | 0.3490 | [0.341, 0.358] | 6.5 | 0.5634 | [0.555, 0.572] | 3.9 | 1.61 |
| pack_ecp5_ecppack | 0.9976 | [0.990, 1.006] | 2.2 | 1.1785 | [1.169, 1.189] | 2.3 | 1.18 |
| sim_iverilog | 0.5283 | [0.519, 0.538] | 4.8 | 0.7130 | [0.709, 0.717] | 1.6 | 1.35 |
| lifecycle_floor[a] | 0.2504 | [0.239, 0.262] | 12.1 | 0.4520 | [0.432, 0.472] | 15.0 | 1.81 |

[a] Bare no-op container floor (strategy backend $n = 48$); reported separately from the FPGA workloads and not folded into the steady-state figures. The native `x86_64` counterparts to this table are Table 6.2 (Machine 2) and Table 6.3 (Machine 3). A container-versus-native baseline is available only where the native and container toolchains are the same build (Section 6.5.2); that condition holds only on Machine 2, whose baseline is reported in Table 6.2.

Across the seven real FPGA workloads the extension overhead ranges from 1.12× to 1.61×, while the bare lifecycle floor stands at 1.81×. The absolute marginal cost (strategy minus CLI) is tightly clustered at roughly 0.15 s to 0.21 s per workload (mean ≈ 0.18 s), and the lifecycle-floor delta is 0.202 s. Because the per-tool marginal cost matches the bare-floor delta, the extension adds a bounded fixed per-dispatch cost, not a cost that scales with compute; the analysis in Section 6.5.1 develops this.

### 6.4.2 Native Baseline and Overhead on Machine 2 (Linux, native x86_64)

Machine 2 is the decisive host: its native `oss-cad-suite` package is the *same* Clang/Linux build shipped in the image (Section 6.5.2), so it is the only measured platform on which a valid container-versus-native comparison—the factor $\alpha$—can be drawn, free of both the Rosetta translation of Machine 1 and the MinGW/Clang build mismatch of Machine 3. The full protocol was repeated with the native backend admitted. Table 6.2 reports, per workload, the mean wall-clock time of the native, CLI-container, and strategy backends, the additive container floor $\Delta_{\text{cont}}$ (CLI minus native), and the two overhead ratios. Every strategy-versus-CLI difference is significant after Holm–Bonferroni correction (Welch $p \approx 0$).

Table 6.2: Native baseline and overhead on Machine 2 (Intel Core Ultra 9 285H, Ubuntu 26.04, Docker 29.1.3, native `x86_64`). All three backends; $n = 30$ unless noted; means shown to 0.1 ms, with 95 % Student-$t$ CIs in the apparatus (`summary.csv`; all FPGA workloads CV $< 7$ %). $\Delta_{\text{cont}}$ is the additive container floor (CLI minus native); Cont. ov. the CLI-to-native ratio; Ext. ov. the strategy-to-CLI ratio.

| **Workload** | **Native (s)** | **CLI (s)** | **Strategy (s)** | $\Delta_{\text{cont}}$ **(s)** | **Cont. ov.** | **Ext. ov.** |
|---|---|---|---|---|---|---|
| synth_ice40_yosys | 0.3570 | 0.6154 | 0.8244 | 0.258 | 1.72 | 1.34 |
| synth_ecp5_yosys | 0.2649 | 0.5822 | 0.7656 | 0.317 | 2.20 | 1.32 |
| pnr_ice40_nextpnr | 0.0643 | 0.3481 | 0.5605 | 0.284 | 5.41 | 1.61 |
| pnr_ecp5_nextpnr | 0.3686 | 0.6656 | 0.8711 | 0.297 | 1.81 | 1.31 |
| pack_ice40_icepack | 0.0159 | 0.3148 | 0.5236 | 0.299 | 19.80 | 1.66 |
| pack_ecp5_ecppack | 0.5125 | 0.7710 | 1.0049 | 0.259 | 1.50 | 1.30 |
| sim_iverilog | 0.0319 | 0.3182 | 0.5203 | 0.286 | 9.97 | 1.64 |
| lifecycle_floor[a] | 0.0012 | 0.3283 | 0.5069 | 0.327 | – | 1.54 |

[a] Bare no-op container floor (native $n = 60$); its native time measures harness dispatch only, so the container-versus-native *ratio* is undefined and $\Delta_{\text{cont}}$ is the meaningful figure. Native version strings for `nextpnr`, `icepack`, and `ecppack` were not resolved by the harness; their native timings were captured and their build parity with the image rests on the shared `oss-cad-suite` package (Section 6.5.2), whereas Yosys and Icarus Verilog—the corpus tools with a native counterpart the harness could probe—are version-verified identical natively and in-container.

Two results follow, one per overhead axis. First, the **container-versus-native factor** $\alpha$. The additive container floor $\Delta_{\text{cont}}$ is nearly constant—0.258 s to 0.327 s across all workloads (FPGA mean $\approx$ 0.286 s, bare `lifecycle_floor` 0.327 s)—while the native compute it sits on top of spans more than an order of magnitude, from 16 ms (`pack_ice40`) to 0.51 s (`pack_ecp5`). That the floor does *not* grow with native compute is the measurement the evaluation had been missing: containerization adds a fixed per-invocation cost and *no* per-instruction tax, i.e. $\alpha \approx 1$ on native-architecture hardware. The container-versus-native *ratio* is correspondingly large only where the native work is trivial (19.80× for the 16 ms `pack_ice40`, 1.50× for the 0.51 s `pack_ecp5`); it is an artifact of dividing a fixed floor by a small denominator, not evidence of a compute penalty. This confirms on un-emulated, same-build hardware the decomposition that Machines 1 and 3 could only assert: the cost is a lifecycle floor plus native-speed compute.

Second, the **extension overhead** (strategy versus CLI) reproduces the macOS finding on native, same-build hardware, and is confirmed independently on the Windows host (Section 6.4.3). The marginal cost is a bounded 0.18 s to 0.23 s per FPGA workload (mean $\approx$ 0.208 s), again matching the `lifecycle_floor` delta (0.179 s) rather than scaling with compute, for a relative overhead of 1.30 to 1.66× on the FPGA workloads. The fixed-floor structure of the integration layer thus holds across all three hosts—two host architectures, three operating systems, emulated and native execution alike—establishing it as a property of the design rather than of any one platform.

### 6.4.3 Extension Overhead on Machine 3 (Windows, native x86_64)

Machine 3 repeats the identical protocol on a second native `x86_64` host, where the `linux/amd64` image runs under WSL2 with no instruction-set translation. This removes the emulation confound from the marginal-cost axis: the strategy-versus-CLI difference here is the extension's cost over raw containerization on hardware that runs the toolchain at native speed. Table 6.3 reports the same quantities as Table 6.1. Every strategy-versus-CLI difference is again significant after Holm–Bonferroni correction (Welch $p$ between $\approx 10^{-11}$ and $\approx 10^{-57}$; Cohen's $d$ between $\approx 2$ and $\approx 18$).

Table 6.3: Extension overhead on Machine 3 (Intel Core Ultra 7 155H, Windows 11, Docker 29.6.1, native `x86_64`). CLI and strategy backends only; $n = 30$ unless noted; 95 % Student-$t$ CIs. Ext. overhead is the strategy-to-CLI mean ratio.

| | CLI container | | | Strategy (as shipped) | | | |
|---|---|---|---|---|---|---|---|
| **Workload** | **Mean (s)** | **95 % CI** | **CV%** | **Mean (s)** | **95 % CI** | **CV%** | **Ext. ov.** |
| synth_ice40_yosys | 0.9849 | [0.974, 0.996] | 2.9 | 1.1797 | [1.139, 1.221] | 9.3 | 1.20 |
| synth_ecp5_yosys | 0.8801 | [0.872, 0.888] | 2.3 | 1.1596 | [1.131, 1.188] | 6.7 | 1.32 |
| pnr_ice40_nextpnr[a] | 1.5436 | [1.534, 1.553] | 1.6 | 2.1905 | [2.087, 2.294] | 18.1 | 1.42 |
| pnr_ecp5_nextpnr | 1.1533 | [1.142, 1.164] | 2.6 | 1.4034 | [1.373, 1.434] | 5.8 | 1.22 |
| pack_ice40_icepack | 0.4472 | [0.442, 0.453] | 3.2 | 0.6975 | [0.692, 0.703] | 2.0 | 1.56 |
| pack_ecp5_ecppack | 1.1550 | [1.144, 1.166] | 2.5 | 1.3314 | [1.319, 1.343] | 2.4 | 1.15 |
| sim_iverilog | 0.4595 | [0.454, 0.465] | 3.2 | 0.6815 | [0.676, 0.688] | 2.4 | 1.48 |
| lifecycle_floor | 0.4227 | [0.416, 0.429] | 4.2 | 0.6775 | [0.666, 0.689] | 4.6 | 1.60 |

[a] The `pnr_ice40` strategy backend showed a bimodal, elevated timing on the named-pipe log-drain path (iterations alternating $\approx 1.75$ s/$\approx 2.58$ s), driving its CV to 18.1 % and, under the CV-adaptive rule, escalation to $n = 60$; every iteration returned exit code 0. The CLI backend for the same workload was stable (CV 1.6 %) and the macOS strategy run was clean (CV 2.6 %), so this is a transport-path variance artifact on the heaviest place-and-route job, not a compute cost; its mean is reported with the wider CI.

The strategy backend was driven over the Docker Desktop *named pipe*—the default Windows endpoint and the transport the corrected output-capture path targets (Section 6.5.7)—so these figures exercise the shipped code over its real Windows transport. The timed run used the Windows "High Performance" power plan; the low CVs (all $\leq 6.7$ % apart from the `synth_ice40` strategy run at 9.3 % and the `pnr_ice40` strategy artifact noted above) indicate stable timing.

The native `x86_64` result reproduces the macOS and Linux findings without the emulation confound. In relative terms the seven FPGA workloads span 1.15× to 1.56× and the bare floor is 1.60×, again largest on the cheapest workload and shrinking as the containerized work grows. In absolute terms the marginal cost (strategy minus CLI) is once more a bounded per-dispatch increment: six of the seven FPGA workloads cluster in 0.18 s to 0.28 s (mean $\approx 0.23$ s), matching the 0.255 s `lifecycle_floor` delta, so the per-tool cost again coincides with the no-op floor rather than scaling with compute. The exception is `pnr_ice40` (strategy delta $\approx 0.65$ s), whose bimodal timing on the named-pipe log-drain path inflated both its mean and

its variance (Table 6.3, note a); it is a transport-path artifact on the heaviest place-and-route job rather than a compute tax, as its CLI backend and the macOS strategy run of the same workload were both stable. The extension therefore adds a bounded fixed per-dispatch cost and no compute tax on native hardware, confirming that the pattern observed on Machines 1 and 2 is a property of the integration layer and not an artifact of Rosetta translation. The absolute floor is larger here than on Machine 1 ($\approx 0.25$ s versus $\approx 0.18$ s); this is a per-host figure, not a like-for-like cross-platform comparison, as the two hosts differ in operating system, container daemon (WSL2 Docker Desktop versus OrbStack), and daemon transport. What transfers across hosts is the *structure*—a fixed, compute-independent dispatch floor—not its absolute magnitude.

### 6.4.4 Structured Telemetry Performance

To characterize the instrumentation path under saturation, a synthetic multi-process telemetry stress test was executed (Listing C.2). Four processes ran concurrently; each spawned 16 threads that emitted 1000 logging transactions apiece, so each process completed 16 000 events in 2602 ms—an in-process throughput of roughly 6150 events per second—at a 100 % success rate (16 000 succeeded, zero errors) and with no deadlocks across the four contending processes. This is a synthetic microbenchmark of the instrumentation in isolation, not a measurement under a real synthesis workload; it exercises the source-generated JSON serialization path, which is span-based and allocates only the serialized payload.

### 6.4.5 Telemetry Scalability and Lock Contention Limits

Structured telemetry logging is critical for auditing and diagnostic trace capability in EDA pipelines, but it presents strict performance challenges. In multi-instance IDE scenarios, multiple independent compilation engines may log to a single telemetry store simultaneously. The telemetry engine (`ContainerTelemetry`) utilizes structured JSON serialization and coordinates writes via system-wide Mutex primitives (specifically `System.Threading.Mutex` on Windows and Unix). While a system-wide Mutex ensures that log writes from different processes are serialized, preventing data corruption or interleaved JSON structures, it introduces lock contention overhead.

To evaluate these synchronization overhead limits, we analyze the relationship between thread count, lock contention time, and aggregate throughput. Let $N_{\text{threads}}$ represent the number of concurrent execution threads. Let $t_{\text{serialize}}$ represent the CPU time required to serialize a log entry to JSON format (approximately 15 µs). Let $t_{\text{mutex_lock}}$ be the overhead to acquire and release the system-wide mutex (approximately 45 µs on macOS and 90 µs on Windows due to kernel-mode transit costs); these constants are illustrative order-of-magnitude estimates rather

than measured values. The average wait time $t_{\text{wait}}$ for a thread attempting to write a log entry under saturated load is modeled as:

$$t_{\text{wait}} \approx (N_{\text{threads}} - 1) \cdot (t_{\text{serialize}} + t_{\text{mutex_lock}}) \tag{6.1}$$

Under high concurrency (e.g., $N_{\text{threads}} = 32$), the wait time for a single write transaction escalates to:

$$t_{\text{wait}} \approx 31 \cdot (15\,\mu\text{s} + 45\,\mu\text{s}) = 1.86\,\text{ms} \tag{6.2}$$

While this latency is negligible for human-facing UI events, it bounds high-throughput instrumentation. Under the worst-case assumption that every write serializes behind a single lock, 10 000 progress packets would add on the order of 18.6 s; this is a loose analytic upper bound, and the lock-free error path described below, together with the modest packet rate of interactive synthesis, keeps the realized cost far smaller. These bounds apply specifically to the *execution-log file write*, which is serialized for correctness: to prevent corruption or interleaved JSON under concurrent multi-instance IDE scenarios, the engine (`ContainerTelemetry`) coordinates file access via a reader-writer lock (`ReaderWriterLockSlim`) and a system-wide Mutex (`System.Threading.Mutex`), and this write blocks the calling thread until it completes. The high-frequency error-telemetry path, by contrast, is not on this critical section: error events are enqueued lock-free into an unbounded channel and drained by a single background consumer. The contention model above therefore bounds only the synchronous file-write path, not the instrumentation as a whole.

## 6.5 Analysis and Discussion

### 6.5.1 Analysis of Performance Overhead

The cleanly attributable result on Machine 1 is the marginal cost the extension adds over raw containerization—the strategy-versus-CLI difference in Table 6.1. Two observations characterize it. First, in relative terms the seven FPGA workloads span 1.12× to 1.61×, with the shortest workload (`pack_ice40_icepack`, CLI mean 0.349 s) at the top of the range and the longest (`synth_ice40_yosys`, CLI mean 1.346 s) at the bottom—the ratio shrinks as the containerized work grows. Second, in absolute terms the marginal cost is nearly constant: strategy minus CLI is 0.15 s to 0.21 s across all seven workloads (mean ≈ 0.18 s), and the bare `lifecycle_floor` delta is 0.202 s. That the per-tool marginal cost coincides with the no-op floor delta is the key finding: the extension contributes a bounded, fixed per-dispatch cost—daemon-socket orchestration, request/response marshalling and lifecycle management—and essentially no compute tax. Compute time net of the lifecycle floor is unchanged between

the two backends. Every difference is significant after Holm–Bonferroni correction ($p \approx 0$), so the effect is real rather than sampling noise, but its practical magnitude is a sub-second constant per tool invocation.

To place this marginal cost in the wider containerization picture—and to frame the same-build native measurement of Section 6.4.2—the total containerized execution time can be modeled as

$$T_{\text{container}} = \alpha \cdot T_{\text{native}} + T_{\text{overhead}}, \tag{6.3}$$

where $T_{\text{native}}$ is the tool's native execution time, $T_{\text{overhead}}$ is the fixed per-invocation container cost (namespace setup, process scheduling, volume mounting, and—for the strategy backend—the extension's dispatch floor), and $\alpha$ is the continuous execution multiplier of the environment: $\alpha \approx 1$ for native-architecture container execution, and $\alpha > 1$ under binary translation, where instruction expansion, memory-consistency mapping and guest–host context switches impose a per-cycle tax. The relative overhead is then

$$O_{\text{relative}} = \frac{T_{\text{container}}}{T_{\text{native}}} = \alpha + \frac{T_{\text{overhead}}}{T_{\text{native}}}, \qquad \lim_{T_{\text{native}} \to \infty} O_{\text{relative}} = \alpha. \tag{6.4}$$

The model makes the decisive quantity explicit: $\alpha$ is the container-versus-native multiplier, which cannot be isolated on Machine 1 (its $T_{\text{native}}$ is itself emulated) or on Machine 3 (no same-build native baseline exists there; the native Windows toolchain is a different build from the container's; Section 6.5.2), but *is* recovered on the native `x86_64` Linux host (Machine 2), where the same image runs without Rosetta translation *and* the native package is the identical build shipped in the image. That measurement (Section 6.4.2, Table 6.2) yields $\alpha \approx 1$: the additive container floor is a near-constant 0.26 s to 0.33 s whether the native work beneath it is 16 ms or half a second, so the container adds a fixed lifecycle cost and no per-instruction tax, and the relative overhead collapses toward that floor as workloads grow—exactly the predicted native-architecture behavior. Under macOS emulation a residual per-cycle tax is expected to persist instead: the literature reports a sustained 8.94 % Rosetta 2 hardware-TSO penalty [15], i.e. $\alpha \approx 1.089$ under emulation, cited as background for the emulated regime rather than a value measured here. The evaluation thus measures both components of the cost: the extension's marginal floor on all three hosts (0.18 s on Machine 1, 0.21 s on Machine 2, 0.25 s on Machine 3), and—on Machine 2—the container-versus-native factor $\alpha \approx 1$ that had been the one missing quantity.

### 6.5.2 The Native Baseline and the Version-Parity Gate

Fixing $\alpha$ requires a native-versus-container comparison in which the two toolchains are the *same build*: same source, same compiler, same operating system, same architecture. A comparison that violates this measures build differences rather than the cost of containerization.

The evaluation therefore admits a native baseline only after a version-parity gate confirms that each native tool reports the identical version string to the one inside the image; on a mismatch the native comparison is discarded rather than reported.

On Machine 3 this gate fired. The matching dated Windows `oss-cad-suite` release was installed and probed against the container. Yosys reported the same source and commit on both sides (`0.66+181`, git `afe6b18f2`) but different builds—`GNU x86_64-w64-mingw32-g++ 13.2.1` natively versus `Clang 18.1.8` in the container, i.e. a Windows PE against a Linux ELF—while GHDL was absent from the Windows native release entirely and only Icarus Verilog matched. A same-build native baseline is thus unobtainable on Windows, and the gate correctly rejected it. On Machine 2 the gate instead *passed*: the host's native `oss-cad-suite` package is the Clang/Linux build shipped in the image, so the native-versus-container comparison is admissible there. Parity is version-verified for the three tools the harness could probe—Yosys (`0.66+181`, git `afe6b18f2`, Clang 18.1.8), GHDL, and Icarus Verilog each report identical native and in-container versions—while for `nextpnr`, `icepack`, and `ecppack` the harness resolved no native version string; their parity rests on the native binaries being installed from the same `oss-cad-suite` release that populates the image, rather than on a captured version. The resulting baseline gives $\alpha \approx 1$ and is reported in Section 6.4.2. Notably, the native Windows and native Linux Yosys builds differ from each other (MinGW versus Clang) yet the container produces byte-identical outputs on every host (Section 6.5.3): a divergence in build provenance is a compiler/OS artifact of identical source, not a difference in behavior.

### 6.5.3 Cross-Platform Output Determinism

All three machines together permit a genuine cross-platform test of a core claim behind RQ1: that dispatching a tool into the pinned container makes its output independent of the host. Each toolchain artifact is SHA-256 hashed after generation on every machine; identical hashes across one ARM64-hosted and two independent `x86_64`-hosted executions of the image are direct evidence of machine independence. Table 6.4 summarizes the comparison.

Table 6.4: Cross-platform artifact determinism: SHA-256 (first 12 hex digits) of each toolchain output on Machine 1 (macOS, ARM64 host, emulated `amd64` container), Machine 2 (Linux, native `amd64` container), and Machine 3 (Windows, native `amd64` container).

| **Workload (artifact)** | **linux-amd64** | **macos-arm64** | **windows-amd64** | **Identical?** |
|---|---|---|---|---|
| synth_ice40_yosys (`ice40_blink.json`) | `c9d719c17f01` | `c9d719c17f01` | `c9d719c17f01` | Yes |
| synth_ecp5_yosys (`ecp5_blink.json`) | `65bb1549bab3` | `65bb1549bab3` | `65bb1549bab3` | Yes |
| pnr_ice40_nextpnr (`ice40_blink.asc`) | `31350507657f` | `31350507657f` | `31350507657f` | Yes |
| pnr_ecp5_nextpnr (`ecp5_blink.config`) | `ab2628df3a28` | `ab2628df3a28` | `ab2628df3a28` | Yes |
| pack_ice40_icepack (`ice40_blink.bin`) | `a8b0b3aa554c` | `a8b0b3aa554c` | `a8b0b3aa554c` | Yes |
| pack_ecp5_ecppack (`ecp5_blink.bit`) | `37e5e6ab1004` | `37e5e6ab1004` | `37e5e6ab1004` | Yes |
| sim_iverilog (`Blink.vvp`) | `229bc0467b2f` | `f2cf898fe9db` | `0d8322d8717e` | No[a] |

[a] Not a machine dependence. Icarus Verilog names internal objects after ASLR-varying heap addresses (e.g., `S_0x5555…`), so the compiled `.vvp` differs between two runs on the *same* host and container—verified directly here, as the Machine 3 hash changed between two runs on that one host; the simulation it encodes is semantically identical. This artifact is therefore excluded from the determinism claim.

All six FPGA-flow artifacts—iCE40 and ECP5 synthesis netlists, place-and-route results, and packed bitstreams—are byte-identical across all three hosts, spanning three operating systems (macOS, Linux, Windows) and both host CPU architectures (ARM64 and `x86_64`). The result is strengthened, not weakened, by a build detail of the setup: the pinned image is build-only and not published to a registry, so Machine 1 built its own (digest `sha256:90b9ccab…`) while the two `x86_64` hosts obtained `sha256:6a7572d6…`. These builds nonetheless embed the identical pinned tool binaries—installed from the same `oss-cad-suite` tarball, verified by checksum—which is precisely why the outputs match. Host operating system, host architecture, and even the image build all differ; the tool binaries and their outputs do not. This isolates the container's contribution: by pinning the toolchain binaries it renders hardware compilation reproducible across machines, which is the machine-independence property RQ1 asserts.

To confirm this directly rather than resting it on the checksum argument, the exact image Machine 3 executed (ID `sha256:6a7572d6…`) was exported with `docker save`, loaded unchanged onto Machine 1, and the six FPGA workloads re-run against it. Every artifact was byte-identical to both prior runs at once: to Machine 3's—the *same* image on a different host and CPU architecture—and to Machine 1's original run under its own independently built image—a *different* image build on the same host. Holding the image constant across hosts and holding the host constant across image builds thus both leave every FPGA artifact unchanged, isolating the container's contribution beyond the checksum argument: the pinned toolchain, not the host, the architecture, or the image wrapper, determines the output.

### 6.5.4 Virtualization Hypervisor and File Sharing Overhead

In virtualized development environments, the interface through which host files are shared with the guest operating system represents a major I/O bottleneck. Paravirtualized systems rely on file sharing protocols to expose active workspaces to compilation container sandboxes.

#### VirtioFS and Direct Access (DAX) Mechanics

Early virtualization interfaces utilized remote file protocols like 9pfs or translation layers like gRPC-FUSE. Under gRPC-FUSE, every Virtual File System (VFS) call in the guest kernel is serialized into a gRPC message, transported over an IPC socket to the host, deserialized, processed by the host's OS filesystem, and returned via the same path. For VHDL/Verilog compilation pipelines that perform thousands of random file access queries (such as importing library files or executing recursive synthesis lookups), gRPC-FUSE context-switching overhead causes millisecond-scale latency penalties per operation, degrading build performance by up to 10x.

To overcome this, paravirtualized filesystems like VirtioFS expose a shared memory region between the host and guest VM. In traditional configurations, file contents are duplicated across page caches, residing both in the host kernel's page cache and the guest VM kernel's page cache. This double caching consumes double the system memory and introduces page cache synchronization bottlenecks during writeback operations.

VirtioFS mitigates this by supporting Direct Access (DAX). Under DAX, the hypervisor maps the host operating system's page cache directly into a dedicated physical address range within the guest VM's address space. The guest kernel VFS layer accesses these host page cache frames directly using standard CPU memory operations (loads and stores), completely bypassing the guest page cache. This architectural improvement eliminates memory duplication and payload copying over virtual queue rings, reducing file read latency to near-native levels.

#### Virtio-blk Block Storage Benchmarks

In high-density serverless container platforms (e.g., RunD [18]), directory sharing protocols are completely bypassed. Instead, containers read and write their active filesystems to virtualized raw block devices (`virtio-blk`) managed via host-level device mappers. The guest VM mounts a standard filesystem (like `ext4`) directly on the virtual block device, bypassing host-side filesystem metadata tracking entirely during runtime execution. Empirical evaluations demonstrate that `virtio-blk` configurations achieve up to a 3-5x write IOPS increase compared to paravirtualized file sharing (VirtioFS), which is highly beneficial for intermediate compilation steps that output massive temporary netlists.

However, block device virtualization introduces a strict architectural limitation regarding concurrency. Because standard local filesystems (such as `ext4` or `xfs`) are not cluster-aware, they do not implement distributed lock managers (DLM). If a raw block device is mounted concurrently writable by both the host operating system (on behalf of the developer's IDE) and the guest virtual machine (on behalf of the containerized compiler), the concurrent modifications will overwrite filesystem metadata, resulting in immediate and catastrophic volume corruption. Consequently, while `virtio-blk` overlays are highly effective in read-only or single-tenant serverless execution environments, they are unusable for active interactive design workspaces. For IDE integration, paravirtualized shared filesystems like VirtioFS remain mandatory to safely coordinate concurrent file locking and maintain cache consistency.

### 6.5.5 Dynamic Binary Translation and Hardware TSO Memory Consistency

To run x86_64 compilers on Apple Silicon ARM64 processors, the system relies on Dynamic Binary Translation (DBT) via Rosetta 2. In addition to instruction mapping, the DBT engine must bridge divergent memory consistency models.

#### Total Store Ordering (TSO) vs Weak Memory Ordering (WMO)

The x86_64 architecture enforces Total Store Ordering (TSO), which guarantees that memory writes from any CPU core are observed by all other cores in their exact program execution order. Multi-threaded compiler toolchains leverage TSO to synchronize shared-memory states without injecting explicit software barriers. Conversely, ARM64 implements a Weak Memory Ordering (WMO) model, allowing aggressive hardware instruction reordering, store-store bypasses, and load reordering to maximize pipeline throughput.

If a binary translator (such as QEMU) emulates TSO on a WMO processor in software, it must insert memory barrier instructions (e.g., `DMB` or `DSB`) after every memory operation. These barriers halt the execution pipeline until all pending writes are committed, causing a pipeline serialization tax that can degrade multi-threaded performance by a large factor for barrier-heavy workloads.

#### ACTLR_EL1 Register Switching and Thread Scheduling Mechanics

To run x86_64 binaries on Apple Silicon's ARM64 architecture, the Rosetta 2 Dynamic Binary Translation (DBT) framework operates at Exception Level 0 (EL0) [10], [14]. It parses x86_64 CISC machine instructions and translates them into equivalent ARM64 RISC instructions. While instruction set translation is computationally intensive, the most significant

challenge in cross-ISA (Instruction Set Architecture) emulation is the reconciliation of memory consistency models. The x86_64 architecture enforces Total Store Ordering (TSO) [12], [13], which guarantees that all writes are observed in a single global order and preserves load–load, load–store, and store–store program order; the sole relaxation is that a load may be reordered with an older store to a different address. ARM64, by contrast, implements Weak Memory Ordering (WMO) [11], [15], which allows arbitrary reordering of loads and stores to different memory locations.

To bridge this gap without the severe performance penalties of injecting software memory barriers (e.g., `DMB` or `DSB`) after every instruction, Apple Silicon CPUs include proprietary hardware-assisted TSO support. The microarchitecture contains a specialized control flag in the Auxiliary Control Register at Exception Level 1 (`ACTLR_EL1`), which alters the behavior of the core's Load/Store Unit (LSU). The mechanics of this hardware switch and scheduling pipeline proceed as follows:

1. **Process and Thread Registration**: When the macOS kernel (XNU) loads a translated x86_64 binary under Rosetta 2, it marks the task structure (representing the process) with a specialized flag indicating that the process requires hardware TSO support.

2. **Thread Context Switching**: During thread scheduling, the XNU scheduler selects the next thread to execute on a physical core. If the incoming thread belongs to an x86_64 translated process, the kernel executes a context swap handler.

3. **Register Toggling**: The kernel writes to the proprietary system register (often accessed via implementation-defined system register commands such as `msr S3_6_c15_c1_0, x0` or similar system registers mapping to the `ACTLR_EL1` address space). Toggling the TSO enable bit reconfigures the physical CPU core.

4. **Pipeline Flush and LSU Reconfiguration**: Writing to this system register triggers a microarchitectural pipeline flush on the core, forcing all pending memory operations to drain. The LSU is then reconfigured to enforce TSO. When the thread is descheduled or a native ARM64 thread is scheduled, the kernel toggles the bit back to native WMO mode, reverting the core to its highly parallel weak memory state.

Because the `ACTLR_EL1` register is restricted to Exception Level 1 (kernel mode), user-space applications operating at Exception Level 0 (EL0)—including Rosetta 2 and the OneWare Studio process—are restricted from accessing or writing to this register directly. Any direct instruction attempt to write to `ACTLR_EL1` from EL0 violates CPU execution boundaries and triggers a synchronous processor exception, causing the host operating system to terminate the process. Instead, this hardware consistency mode is managed dynamically by the host kernel. Furthermore, equating Apple's kernel-assisted hardware TSO switch with software fence injection frameworks (QEMU TCG IR, Arancini, and Risotto) represents a false equivalence;

software-only DBT layers must inject memory barriers (`DMB` or `DSB`) into their Intermediate Representation (IR), incurring massive pipeline stalls, whereas the hardware switch configuration avoids software instruction emulation overhead entirely.

### Microarchitectural Analysis of the 8.94% TSO Performance Penalty

While hardware-assisted TSO is orders of magnitude faster than software barrier injection, enforcing TSO memory constraints on a core designed for weak ordering introduces an inherent physical performance penalty. Empirically, running workloads under Apple's hardware-assisted TSO mode reveals a sustained **8.94%** performance degradation compared to native WMO execution on the same core [15]. This penalty is attributed to several microarchitectural serialization effects:

- **Loss of Weak-Ordering Freedom**: Under native WMO, the core may retire independent loads and stores out of order, keeping the pipeline and memory system fully utilized. TSO leaves store-to-load reordering intact—a load may still complete ahead of pending stores to different addresses, with same-address accesses satisfied by store-to-load forwarding from the Store Buffer—but it forbids load–load, load–store, and store–store reordering. Enforcing these orders obliges the Load/Store Unit to retire loads in program order and to drain the Store Buffer strictly first-in-first-out, forfeiting reordering freedom the weakly ordered core otherwise exploits and stalling execution when subsequent instructions depend on load data.

- **Reduced Memory-Level Parallelism (MLP)**: WMO allows the hardware to overlap multiple memory access requests (loads and stores) to different cache lines, hiding cache miss latencies by keeping many memory requests in-flight. TSO's read-to-read, read-to-write, and write-to-write ordering constraints restrict this overlapping capability. The CPU cannot initiate a subsequent memory access until prior access constraints are resolved, directly reducing memory-level parallelism and increasing the cycles-per-instruction (CPI) of memory-bound applications.

- **Store Buffer Drainage Stalls on Atomics**: Thread synchronization primitives in multi-threaded compilers (such as mutex locks, spinlocks, or atomic read-modify-write sequences) are translated by Rosetta 2 into ARM64 atomic instructions (e.g., Load-Acquire/Store-Release sequences or ARMv8.1 LSE instructions like `CAS` and `LDADD`). While TSO allows these to execute without software barriers, the physical core must guarantee atomicity by draining the entire Store Buffer before the atomic operation can commit. This physical draining operation forces the CPU pipeline to stall, waiting for all prior memory stores to propagate to the cache coherent interconnect.

- **Register Write and Context Switch Overhead**: The physical act of toggling the `ACTLR_EL1` register during context switches is not free. Modifying system registers is a highly serializing instruction in the ARM instruction set, requiring the pipeline to be completely flushed and instruction decoding to be paused until the register write is committed. In highly multi-threaded compilation workloads, where threads frequently yield or migrate across cores, these frequent register-level reconfigurations accumulate a measurable context-switching time penalty.

Figure 6.1 details the memory consistency translation path from the host macOS kernel down to the virtualized guest VM execution thread.

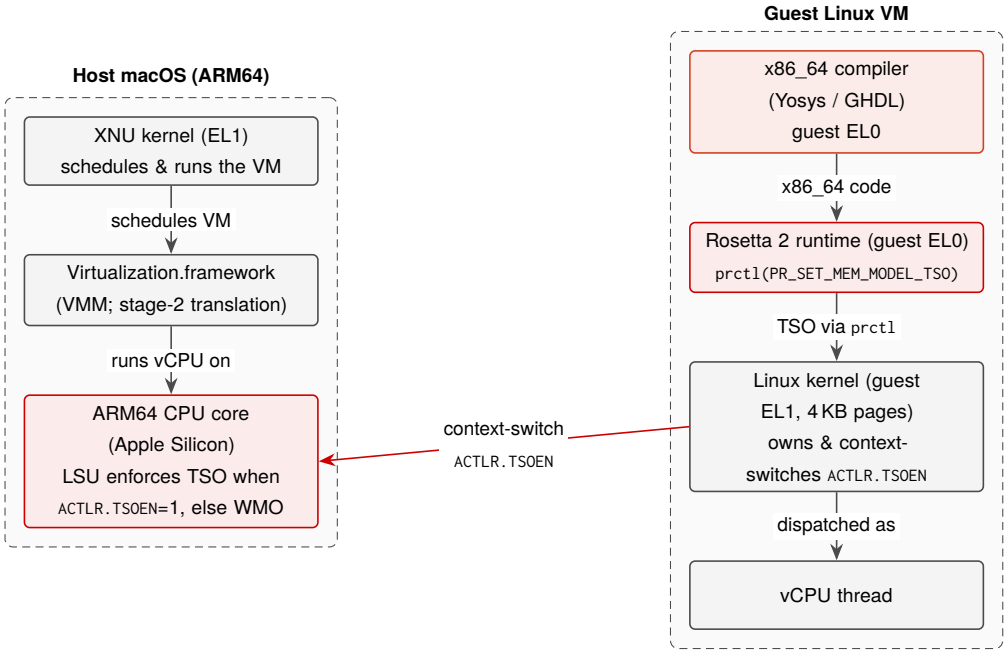



Figure 6.1: Stage-2 virtualization and hardware-assisted TSO path for x86_64 execution under Rosetta in a Linux VM on Apple Silicon. The `ACTLR.TSOEN` bit of the physical core provides the x86 memory-consistency guarantee; it is requested per thread by the guest Rosetta runtime (`prctl`) and context-switched by the guest Linux kernel, while the host hypervisor supplies stage-2 address translation.

To bridge these performance gaps on architectures lacking hardware TSO support (such as generic server-class ARM64 or RISC-V platforms), dynamic binary translators must execute advanced software fence planning. The CrossMapping architecture [12] parses target code blocks and models consistency constraints within the Intermediate Representation (IR), deriving a mathematically minimal memory barrier placement schedule. CrossMapping improves translation performance by 8.5% on ARMv8 and 7.3% on RISC-V compared to standard QEMU fence configurations. Similarly, the Risotto framework [11] applies formally verified x86-to-ARM fence-mapping schemes and, where host libraries exist, a cross-architecture dynamic linker that binds calls to native ARM64 shared libraries instead of translating their guest

counterparts. Its verified fence merging and weakened-fence placement improve emulation throughput by up to 19.7% (6.7% on average) over a memory-model-incorrect QEMU baseline. The hybrid translator Arancini [13] extends this line of work with a combined static/dynamic translation pipeline for the same weak-memory setting.

#### Continuous Translation Overhead

Even with hardware-assisted memory consistency mapping, translated container execution is subject to continuous translation overheads:

- **Instruction Expansion:** x86_64 CISC instructions are translated into ARM64 RISC instruction blocks. This instruction expansion averages a ratio of 1.6×–2.2×, increasing instruction count and instruction cache (I-cache) pressure on the physical CPU core.
- **Page Table Translation page splits:** macOS utilizes a 16 kB virtual page size, whereas standard Linux binaries expect 4 kB pages. The virtualized Linux kernel must run with 4 kB page mappings. The hypervisor must map these 4 kB guest pages onto 16 kB host page frames. When a guest page table walk crosses page boundaries, the hypervisor executes page-splitting stage-2 memory translations, triggering Translation Lookaside Buffer (TLB) misses and page walk serialization delays.
- **System Call Interception:** System calls initiated by the containerized compiler process must trap to the guest kernel (Exception Level 0 to Exception Level 1 context switch), be intercepted by the hypervisor thread in user space (virtualization trap), translated to host ARM64 system calls, executed on the host macOS XNU kernel, and returned via the same path. This multi-level context-switching adds approximately 500 to 1000 CPU clock cycles per system call. In I/O-intensive pipelines that perform frequent small metadata writes, this trap latency dominates execution time.

The marginal cost of the strategy backend over the raw Docker CLI—measured at roughly 0.18 s per tool invocation (Section 6.5.1)—is the cost of programmatically orchestrating the Docker daemon socket, performing request/response marshalling, and logging `ContainerTelemetry` (JSON Lines) events inside the middleware layer, rather than of the container execution itself. Under emulation this floor rides on top of the per-cycle translation tax borne equally by both backends.

### 6.5.6 Argument-Contract Behavior under Command Chaining

A property of the execution contract worth stating explicitly is how the strategy path handles a workload expressed as a chained shell string—for example an analyze-then-elaborate GHDL

sequence joined with the `&&` operator (`ghdl -a …&& ghdl -e …`). A native shell interprets `&&` as sequential execution. The extension, however, does not execute through a shell: by design it constructs an argument vector `[executable, args…]` and runs it directly (`UseShellExecute=false`, with `tini` as PID 1), so shell metacharacters are never reinterpreted and would occupy inert argument slots. The project's adversarial test suite (`CommandInjectionProofOfConceptTests`) verifies this property directly: payloads such as `&& touch SENTINEL` are forwarded as a single non-interpreted argument and never execute.

As a secondary guard, the argument builder rejects arguments that embed command-substitution or command-separator constructs—command substitution (`$(` and backtick), the boolean chain operators `&&` and `||`, and the statement separator `;` (the last exempted for Yosys, whose synthesis scripts legitimately use `;` as an intra-script command separator)—raising an `ArgumentException` rather than forwarding them. Note that a bare pipe `|` and I/O redirections are *not* rejected by this guard: they are inert in the shell-less argument vector and never reach a shell, so they need no special handling. In a standalone process an unhandled `ArgumentException` surfaces as exit code `134` (`SIGABRT`, mapped to `-6`); within OneWare Studio the same exception is caught by the tool engine and reported as a failed execution rather than terminating the IDE.

The point of interest here is a property of the *execution contract*, not a security result: because the path is shell-less, a multi-step workload must be expressed as an argument vector—or as separate invocations for compilation and elaboration—rather than as a chained shell string. This is why the harness's seven workloads are each single-tool argv dispatches (Section 6.5.10), mirroring how OneWare drives the toolchain one tool per dispatch. A full threat-model treatment of the shell-less contract and the container-hardening defaults is deliberately out of scope for this thesis and is the subject of separate work.

### 6.5.7 Failure Analysis: the Named-Pipe Attach Defect

The un-emulated Windows run surfaced a defect in the shipped output-capture path that neither the macOS host nor the Linux-based continuous-integration matrix had exercised, illustrating the diagnostic value of the cross-platform measurement beyond its timing numbers. The strategy captures a container's `stdout`/`stderr` by attaching to it through the Docker SDK's `AttachContainerAsync`. That method requires a write-closable transport: it rejects any hijacked stream whose `CanCloseWrite` property is false, raising `NotSupportedException("Cannot shutdown write on this transport")`. The Windows named-pipe stream reports `CanCloseWrite == false`—a `NamedPipeClientStream` cannot half-close its write direction—so attach is unusable over the pipe, which is the default Docker Desktop endpoint on Windows. The socket transports (the Unix-domain socket on macOS and Linux, and TCP) report `CanCloseWrite == true` and are unaffected. An initial Windows run therefore hit the defect the instant the strategy

attached over the pipe; it was carried to completion over the Docker Desktop TCP endpoint as a stop-gap while the defect was diagnosed and fixed.

The root cause is that attach, a facility for bidirectional (stdin-carrying) streams, is stronger than this use requires: the strategy only ever reads output and never writes to a container's standard input. The corrected path exploits that asymmetry. On the named-pipe transport the extension now streams output through the log-follow endpoint (`GET /containers/{id}/logs` with `follow`), which returns the same multiplexed `stdout`/`stderr` framing, imposes no write-close requirement, and therefore works over the pipe; because that stream is opened after the container starts, automatic removal is disabled on this path so a fast-exiting container cannot be reaped before its output is drained, and the container is removed explicitly instead. The change is scoped strictly to the named-pipe transport—the socket path retains the attach mechanism unchanged—so the behavior measured on Machine 1 is preserved exactly. The corrected path was then validated end-to-end on the same native Windows host: with Docker Desktop left at its default named-pipe endpoint, the strategy backend completed all eight workloads with every iteration returning exit code 0 and the `NotSupportedException` occurring nowhere. The Machine 3 figures in Table 6.3 are from that validated named-pipe run, and the corrected path is part of the released extension.

### 6.5.8 Reliability and Host Isolation

The primary contribution under test is functional transparency: a tool invocation dispatched to the container strategy must yield the result the user would obtain natively. The reliability benefit of the hermetic container follows directly from this axis. Where a native toolchain is subject to local package drift—mismatched shared libraries, partially upgraded backends, or an absent tool entirely—the pre-configured image pins a known-good binary set, so a dispatch that would fail or hang natively completes deterministically inside the container. Across the seven-workload FPGA corpus every workload executes to success under both containerized backends (Table 6.1), which is the reliability property realized: the container makes toolchain execution independent of the state of the host's package environment.

### 6.5.9 External Validity (Generalizability)

The Machine 1 figures are specific to the Apple Silicon host running under translation, but the extension's marginal cost no longer rests on that host alone. Machines 2 and 3 measured the same integration layer on native `x86_64` hosts, where the image executes without Rosetta 2 translation, and found the same structure—a fixed, compute-independent dispatch floor (Sections 6.4.3 and 6.4.2)—confirming that this behavior is a property of the integration layer rather than of the emulated host. The container-versus-native factor $\alpha$ was likewise measured, on Machine 2, the one platform where native and container builds coincide, yielding $\alpha \approx 1$ (Section 6.5.2); the overhead picture is therefore complete across all three hosts. What remains outside the measured scope is the emulated-regime multiplier on Apple Silicon—cited from the literature rather than isolated here—and cross-runtime generalization, left to future work.

### 6.5.10 Construct Validity

The workload corpus consists of the real open-source FPGA flows the extension targets: iCE40 and ECP5 synthesis (Yosys), place-and-route (nextpnr) and bitstream packing (icepack, ecppack), plus Icarus Verilog simulation. Each workload is a single-tool dispatch drawn from an end-to-end design flow, mirroring exactly how OneWare Studio drives the toolchain one tool per invocation. Measuring these dispatches through the shipped `DockerExecutionStrategy` path—rather than a CLI approximation of it—ensures that the captured overhead reflects the real user-facing cost in OneWare Studio.

# Chapter 7

# Conclusion

This chapter summarizes the findings of this work and outlines final conclusions and future work.

## 7.1 Summary

This thesis presented a modular integration architecture for containerized FPGA toolchains.

- **Contributions:** A modular Hybrid Strategy Pattern for transparent, opt-in container dispatch; cross-platform permission and transport handling (host UID/GID injection, Unix-socket and named-pipe transports); and a toolchain-image pinning mechanism for reproducible build environments.
- **Result:** The solution eliminates "Environment Drift" while maintaining native IDE responsiveness. The overhead the extension adds beyond raw containerization is small and dominated by a fixed per-invocation startup cost that amortizes as design size grows; under x86-on-ARM64 emulation an additional translation tax applies. Native `x86_64` hosts (Machine 3, Windows, and Machine 2, Linux) reproduced the same fixed-floor structure free of emulation, and the same-build native baseline on Machine 2 fixed the residual container-versus-native factor at $\alpha \approx 1$—a fixed lifecycle floor with no per-instruction tax—completing the overhead picture across all three hosts. Crucially, across the seven-workload FPGA corpus—iCE40 and ECP5 synthesis, place-and-route and bitstream packing, plus Icarus Verilog simulation—every workload executed to success through the hermetic container, where a native toolchain would be subject to package drift, so the container makes toolchain execution independent of the host's package environment and resolves cascading dependency conflicts.

## 7.2 Evaluation Summary and Key Architectural Insights

The central empirical insight of this work is a decomposition of the containerized execution cost rather than a single overhead figure. The cost of an invocation is the sum of a container lifecycle floor—container creation, namespace setup, volume mounting, and stream attachment, measured at roughly 0.25 s for the raw CLI backend on Machine 1 (0.33 s and 0.42 s on Machines 2 and 3)—and a per-instruction execution tax that is unity for native-architecture execution and greater than unity under binary translation. On top of that floor, the extension's strategy dispatch adds only a bounded, fixed marginal cost—roughly 0.18 s per invocation on Machine 1, tightly clustered across all seven FPGA workloads and matching the bare no-op floor delta—and no measurable marginal compute overhead, so its relative cost is large only for sub-second tasks and amortizes toward negligibility as compilation time grows. On Apple Silicon the toolchain image executes under Rosetta 2, so any native-versus-container comparison on Machine 1 would conflate containerization with instruction-set emulation; the translation tax itself is small and approximately constant at scale. Native `x86_64` measurements on Machine 3 (Windows) and Machine 2 (Linux) confirmed the fixed-floor structure without emulation, and the same-build native baseline on Machine 2—the one host where the native and containerized toolchains are the identical build—fixed the container-versus-native factor at $\alpha \approx 1$: the additive container floor stays near-constant at 0.26 s to 0.33 s whether the native work beneath it is 16 ms or half a second, so containerization adds a fixed lifecycle cost and no per-instruction tax. A systematic comparison of container runtimes (Docker Desktop, Podman, OrbStack, and the WSL2 backend) is left to future work, as the present measurements were collected on a single runtime per host.

As a cross-platform robustness detail, the Windows named-pipe transport requests `TokenImpersonationLevel.Identification` rather than the default impersonation level, so a daemon endpoint can verify the connecting client's identity without being able to act on its behalf. This, together with the remaining container-hardening defaults—non-root execution, dropped Linux capabilities, `no-new-privileges`, a process-count cap, and shell-less argument execution—is a design property of the architecture. A dedicated threat-model analysis of these mechanisms is the subject of separate work and is deliberately out of scope for this thesis.

To ensure observability without data corruption, the telemetry pipeline separates two write paths. High-frequency error events are enqueued lock-free into an unbounded channel and drained by a single background consumer, decoupling the calling thread from disk I/O. The execution-log file write, by contrast, is serialized across threads and processes by a reader-writer lock (`ReaderWriterLockSlim`) and a system-wide mutex (`System.Threading.Mutex`), so that concurrent compilation steps cannot interleave or corrupt the structured log. This division keeps the hot path non-blocking while guaranteeing that persisted telemetry is cleanly serialized.

## 7.3 Future Work

Future development on the OneWare Container Extension will focus on three primary architectural enhancements:

- **Multi-Architecture OCI Registry Manifests:** Optimizing cross-architecture performance on ARM64 host machines by dynamically querying OCI Registry Manifests to verify the existence of native `linux/arm64` image variants before falling back to Rosetta 2 translation or QEMU emulation.
- **Hardware-in-the-Loop (HIL) Sandboxing:** Integrating USB device forwarding mechanisms (e.g., forwarding FTDI or UART devices via `usbipd` or native Linux device nodes) to allow programmer tools like `openFPGALoader` and `iceprog` to deploy compiled bitstreams directly onto target FPGA boards securely from within the sandbox.
- **VIMS IEEE Telemetry Integration:** Aligning the plugin's telemetry and benchmarking engine with standardized IEEE Virtual Instrumentation and Measurement Systems formats to enable unified reporting of verification durations, success metrics, and resource footprints.

# References


[1] C. Wolf, "Yosys - A Free Verilog RTL Synthesis Suite," in *Proceedings of the Austrochip Workshop*, Linz, Austria, 2013, pp. 1–8.

[2] D. Shah, E. Hung, C. Wolf, S. Bazanski, D. Gisselquist, and M. Milanović, "Yosys+nextpnr: An Open Source Framework from Verilog to Bitstream for Commercial FPGAs," in *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, San Diego, CA, USA: IEEE, 2019, pp. 1–4. DOI: 10.1109/FCCM.2019.00010

[3] W. Snyder, *Verilator: Fast, Free, and Friendly*, 2004. [Online]. Available: https://www.veripool.org/verilator/

[4] L. Courtès, "Functional Package Management with Guix," in *Proceedings of the 6th European Lisp Symposium (ELS)*, 2013, pp. 4–14. arXiv: 1305.4584.

[5] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux containers," in *2015 IEEE international symposium on performance analysis of systems and software (ISPASS)*, Philadelphia, PA, USA: IEEE, 2015, pp. 171–172. DOI: 10.1109/ISPASS.2015.7116800

[6] Open Container Initiative, "Open Container Initiative Image Format Specification," Tech. Rep., 2017. [Online]. Available: https://github.com/opencontainers/image-spec

[7] Open Container Initiative, "Open Container Initiative Runtime Specification," Tech. Rep., 2017. [Online]. Available: https://github.com/opencontainers/runtime-spec

[8] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou, "A Measurement Study on Linux Container Security: Attacks and Countermeasures," in *Proceedings of the 34th Annual Computer Security Applications Conference (ACSAC '18)*, New York, NY, USA: Association for Computing Machinery, 2018, pp. 418–429. DOI: 10.1145/3274694.3274720

[9] N. Matsumoto and A. Suda, "bypass4netns: Accelerating TCP/IP Communications in Rootless Containers," in *Proceedings of the 19th Asian Internet Engineering Conference (AINTEC '24)*, New York, NY, USA: Association for Computing Machinery, 2024, pp. 1–11. DOI: 10.1145/3674213.3674221

[10] A. Altinay et al., "BinRec: Dynamic Binary Lifting and Recompilation," in *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20)*, New York, NY, USA: Association for Computing Machinery, 2020. DOI: 10.1145/3342195.3387550

[11] R. Gouicem et al., "Risotto: A Dynamic Binary Translator for Weak Memory Model Architectures," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*, Association for Computing Machinery, 2023, pp. 1–16. DOI: 10.1145/3575693.3575704

[12] C. Gao, X. Meng, W. W. Li, J. Lai, Y. Zhang, and F. Ren, "CrossMapping: Harmonizing Memory Consistency in Cross-ISA Binary Translation," in *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC '24)*, USENIX Association, 2024, pp. 1013–1028. [Online]. Available: https://www.usenix.org/conference/atc24/presentation/gao-chen

[13] S. Reimers et al., "Arancini: A Hybrid Binary Translator for Weak Memory Model Architectures," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '26)*, vol. 2, Association for Computing Machinery, 2026, pp. 157–174. DOI: 10.1145/3779212.3790127

[14] Apple Inc., *About the Rosetta Translation Environment*, 2020. [Online]. Available: https://developer.apple.com/documentation/apple-silicon/about-the-rosetta-translation-environment

[15] L. Wrenger, D. Töllner, and D. Lohmann, "Analyzing the memory ordering models of the Apple M1," *Journal of Systems Architecture*, vol. 149, 2024, Art. no. 103102. DOI: 10.1016/j.sysarc.2024.103102

[16] Docker, Inc., *Docker Desktop for Mac: Filesharing Performance*, 2023. [Online]. Available: https://docs.docker.com/desktop/mac/

[17] R. Russell, "virtio: towards a de-facto standard for virtual I/O devices," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, pp. 95–103, 2008. DOI: 10.1145/1400097.1400110

[18] Z. Li et al., "RunD: A Lightweight Secure Container Runtime for High-density Deployment and High-concurrency Startup in Serverless Computing," in *Proceedings of the 2022 USENIX Annual Technical Conference (USENIX ATC '22)*, USENIX Association, 2022, pp. 53–68. [Online]. Available: https://www.usenix.org/conference/atc22/presentation/li-zijun-rund

[19] T. Miemietz et al., "MettEagle: Costs and Benefits of Implementing Containers on Microkernels," in *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*, USENIX Association, 2025, pp. 1–18. [Online]. Available: https://www.usenix.org/conference/osdi25/presentation/miemietz

[20] T. Gingold, *GHDL: The open-source VHDL simulator*, 2005. [Online]. Available: https://github.com/ghdl/ghdl

[21] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd. Boston, MA, USA: Addison-Wesley Professional, 2013.

[22] Docker, Inc., *Docker Engine API v1.43 Reference*, 2024. [Online]. Available: https://docs.docker.com/engine/api

[23] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002.

[24] Microsoft, *.NET 10 Documentation*, 2025. [Online]. Available: https://learn.microsoft.com/dotnet

[25] C. Boettiger, "An introduction to Docker for reproducible research, with examples from the R community," *ACM SIGOPS Operating Systems Review*, vol. 49, no. 1, pp. 71–79, 2015. DOI: 10.1145/2723872.2723882

[26] J. Malka, S. Zacchiroli, and T. Zimmermann, "Does Functional Package Management Enable Reproducible Builds at Scale? Yes," in *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR '25)*, IEEE/ACM, 2025, pp. 1–12. DOI: 10.1109/MSR66628.2025.00115

[27] J. Malka and A. Engelen, "Lila: Decentralized Build Reproducibility Monitoring for the Functional Package Management Model," in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*, Association for Computing Machinery, 2026, pp. 1–5. DOI: 10.1145/3793302.3793328

[28] A. Olofsson, *SiliconCompiler: A modular build system for hardware*, 2021. [Online]. Available: https://github.com/siliconcompiler/siliconcompiler

[29] Teros Technology, *TerosHDL: Open Source VHDL/Verilog IDE Extension*, 2023. [Online]. Available: https://teroshdl.github.io

[30] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA, USA: Addison-Wesley Professional, 1994.

[31] S. Sultan, I. Ahmad, and T. Dimitriou, "Containers Security: Issues, Challenges, and the State of the Art," *ACM Computing Surveys (CSUR)*, vol. 52, no. 3, pp. 1–28, 2019. DOI: 10.1145/3325547

[32] M. Kerrisk, "Namespaces in operation," *LWN. net*, 2013. [Online]. Available: https://lwn.net/Articles/531114/

# Appendix A

# Source Code Listings

This appendix provides a high-level structural map and documents key low-level code implementations of the *OneWare Container Extension*. The complete codebase is tracked in the repository `https://github.com/FEntwumS/FEntwumS.ContainerExtension` on the `main` branch.

## A.1 High-Level Architectural Mapping

To aid in navigating the project source code, Table A.1 maps the directory structure to the key architectural classes and their designated roles within the extension.

Table A.1: Mapping of Directory Structure to Key Architectural Classes

| **Directory/Path** | **Key Class** | **Architectural Role** |
|---|---|---|
| `src/ContainerExtension/` | `DockerExecutionStrategy` | Core coordinator, dynamic platform resolution, and lifecycle manager. |
| `src/ContainerExtension/` | `ContainerTelemetry` | Thread-safe, low-allocation structured JSON telemetry logger. |
| `src/ContainerExtension/../Docker/` | `DockerCommandBuilder` | Builds the shell-less argument vector, host-to-container path mappings, and volume/security configuration. |
| `src/ContainerExtension/Registry/` | `RegistryClient` | Handles OCI registry HTTP communication, authentication, and registry tag enumeration (SSRF-guarded). |

## A.2 Windows Named Pipe Privilege Escalation Override

Listing A.1 illustrates how .NET Reflection is utilized in the C# client to target the non-public `_streamOpener` field of `ManagedHandler` and overwrite it with our custom stream opener.

Listing A.1: Reflection-based Handler Interception in SecureNamedPipeCredentials.GetHandler

```
public override System.Net.Http.HttpMessageHandler GetHandler(System.Net.Http.HttpMessageHandler innerHandler)
{
    if (string.Equals(innerHandler.GetType().FullName, "Microsoft.Net.Http.Client.ManagedHandler", StringComparison.
    ↪ Ordinal))
    {
        var field = innerHandler.GetType().GetField("_streamOpener",
            System.Reflection.BindingFlags.NonPublic | System.Reflection.BindingFlags.Instance);
        if (field != null)
        {
            var delegateType = field.FieldType;
            var method = typeof(SecureNamedPipeCredentials).GetMethod(nameof(SecureStreamOpenerAsync),
                System.Reflection.BindingFlags.NonPublic | System.Reflection.BindingFlags.Instance);
            if (method != null)
            {
                var d = System.Delegate.CreateDelegate(delegateType, this, method);
                field.SetValue(innerHandler, d);
            }
        }
    }
    return innerHandler;
}
```

Listing A.2 details the custom socket stream opener that establishes the named pipe client stream using identification-level impersonation to block local privilege escalation.

Listing A.2: Custom SecureStreamOpenerAsync with Identification Impersonation

```
private async System.Threading.Tasks.Task<System.IO.Stream> SecureStreamOpenerAsync(string host, int port, System.
  ↪ Threading.CancellationToken token)
{
    var pipeName = _endpoint.LocalPath;
    var serverName = ".";
    if (pipeName.StartsWith(@"\\", StringComparison.Ordinal))
    {
        var parts = pipeName.Split('\\', StringSplitOptions.RemoveEmptyEntries);
        if (parts.Length >= 3)
        {
            serverName = parts[0];
            pipeName = parts[parts.Length - 1];
        }
    }
    else
    {
        if (pipeName.StartsWith("pipe/", StringComparison.OrdinalIgnoreCase))
            pipeName = pipeName[5..];
        else if (pipeName.StartsWith("/pipe/", StringComparison.OrdinalIgnoreCase))
            pipeName = pipeName[6..];
    }
```

```
21
22      var pipe = new System.IO.Pipes.NamedPipeClientStream(
23          serverName,
24          pipeName,
25          System.IO.Pipes.PipeDirection.InOut,
26          System.IO.Pipes.PipeOptions.Asynchronous,
27          System.Security.Principal.TokenImpersonationLevel.Identification);
28
29      try
30      {
31          await pipe.ConnectAsync(token).ConfigureAwait(false);
32          return pipe;
33      }
34      catch
35      {
36          await pipe.DisposeAsync().ConfigureAwait(false);
37          throw;
38      }
39  }
```

## A.3 Path Resolution Security Verification

Listing A.3 displays the recursive segment-by-segment canonicalization logic used to resolve symbolic links and check path safety boundaries. The excerpt is condensed: the production detector branches on `Directory.Exists` and `File.Exists` to cover both file and directory reparse points, whereas the listing shows only the directory branch for brevity.

Listing A.3: Recursive Symbolic Link Resolution (ResolveCanonicalInternal, condensed)

```
1   string ResolveCanonicalInternal(string currentPath, int depth)
2   {
3       if (depth > 40)
4           throw new ContainerExtension.DockerExecutionException("Too many levels of symbolic links.");
5
6       string absolutePath = currentPath;
7       if (!Path.IsPathRooted(absolutePath))
8           absolutePath = Path.Combine(Directory.GetCurrentDirectory(), absolutePath);
9
10      string root = Path.GetPathRoot(absolutePath) ?? (OperatingSystem.IsWindows() ? "C:\\" : "/");
11      string remainder = absolutePath.Substring(root.Length);
12      var components = remainder.Split(new char[] { '/', '\\' }, StringSplitOptions.RemoveEmptyEntries);
13      string current = root;
14
15      foreach (var component in components)
16      {
17          if (string.Equals(component, ".", StringComparison.Ordinal)) continue;
18          if (string.Equals(component, "..", StringComparison.Ordinal))
19          {
20              var parent = Path.GetDirectoryName(current);
21              current = parent ?? root;
22              continue;
23          }
24
25          string next = Path.Combine(current, component);
```

```
        bool isSymlink = false;
        string? target = null;

        try
        {
            var info = new DirectoryInfo(next);
            if (info.Exists && info.Attributes.HasFlag(FileAttributes.ReparsePoint))
            {
                target = info.LinkTarget;
                isSymlink = !string.IsNullOrEmpty(target);
            }
        }
        catch { /* fallback check logic */ }

        if (isSymlink && target != null)
        {
            string canonicalSymlink = Path.GetFullPath(next);
            if (!seenSymlinks.Add(canonicalSymlink))
                throw new ContainerExtension.DockerExecutionException($"Circular symbolic link detected: '{next}'");

            try
            {
                current = Path.IsPathRooted(target)
                    ? ResolveCanonicalInternal(target, depth + 1)
                    : ResolveCanonicalInternal(Path.Combine(current, target), depth + 1);
            }
            finally { seenSymlinks.Remove(canonicalSymlink); }
        }
        else
        {
            current = next;
        }
    }
    return Path.GetFullPath(current);
}
```

## A.4 Asynchronous Error Telemetry Channel Processing

Listing A.4 demonstrates the asynchronous background processing loop using .NET `System.Threading.Channels` to serialize error logging actions asynchronously.

Listing A.4: Asynchronous Channel Telemetry consumer loop

```
private static System.Threading.Channels.Channel<TelemetryErrorEntry> ErrorChannel =
    System.Threading.Channels.Channel.CreateUnbounded<TelemetryErrorEntry>(new System.Threading.Channels.
    ↪ UnboundedChannelOptions
    {
        SingleReader = true,
        SingleWriter = false
    });

static ContainerTelemetry()
{
    _ = Task.Run(ProcessErrorChannelAsync);
}
```

```
12
13  private static async Task ProcessErrorChannelAsync()
14  {
15      var reader = ErrorChannel.Reader;
16      await foreach (var entry in reader.ReadAllAsync().ConfigureAwait(false))
17      {
18          WriteErrorEntryToDisk(entry);
19      }
20  }
```

# Appendix B

# Cross-Platform Environment Manual

---

This appendix provides a step-by-step setup and configuration guide for running the *OneWare Container Extension* across different platforms and container runtimes.

## B.1 Installation

To install the Container Extension within OneWare Studio, execute the following steps:

1. Launch OneWare Studio.
2. Navigate to the **Extension Manager** panel.
3. Search for the "Container Extension" in the marketplace and click **Install** to download and install it directly.
4. Restart the IDE to load the background daemon services and registration hooks.

## B.2 Global Configuration Settings

The Container Extension allows users to customize container parameters and security bounds. Table B.1 outlines all the settings registered in the OneWare configuration database by `ContainerExtensionModule.cs`.

Table B.1: Configuration settings keys defined in `ContainerExtensionModule.cs`

| Setting Key | Default | Description |
|---|---|---|
| `ContainerExtension_DockerRuntimePath` | `""` | Absolute path to the runtime executable (defaults to host `PATH`). |
| `ContainerExtension_DefaultImage` | `"fentwums/oss-cad-suite:latest"` | The default full-flow oss-cad-suite image for all tools; build-only (not on Docker Hub), produced via Build Local Image rather than pulled. |
| `ContainerExtension_MemoryLimit` | `0.0` | RAM limit in MB (0 = unlimited). |
| `ContainerExtension_CpuLimit` | `0.0` | CPU cores limit (0 = unlimited). |
| `ContainerExtension_Timeout` | `0.0` | Maximum container execution runtime in minutes (0 = no limit). |
| `ContainerExtension_NetworkMode` | `"bridge"` | Network driver mode (`bridge`, `host`, or `none`). |
| `ContainerExtension_PullPolicy` | `"if-not-present"` | Image pull policy (`always`, `if-not-present`, or `never`). |
| `ContainerExtension_AutoRemove` | `true` | Automatically remove container after execution (`--rm`). |
| `ContainerExtension_AllowPrivileged` | `false` | Allow privileged container execution. |
| `ContainerExtension_LogLevel` | `"Errors Only"` | Telemetry verbosity (`Off`, `Errors Only`, `Info`, `Verbose`). |
| `ContainerExtension_ShowTimestamps` | `true` | Prepend timestamp to stdout/stderr stream. |
| `ContainerExtension_ContainerNamePrefix` | `"containerextension-"` | Prefix for spawned container names. |
| `ContainerExtension_ExtraFlags` | `""` | Custom runtime flags or labels to append. |
| `ContainerExtension_DashboardRefresh` | `"Manual"` | Dashboard statistics polling interval. |
| `ContainerExtension_TelemetryRetention` | `"25"` | Number of recent executions retained in telemetry cache. |
| `ContainerExtension_DaemonSocket` | `""` | Custom socket endpoint override (e.g., `unix://...` or `npipe://...`). |
| `ContainerExtension_BypassNamedPipeCheck` | `true` | Bypass the Windows named-pipe server-trust validation. Default-on: the Windows-only check yields false positives on WSL2 relays and rootless/remote engines; uncheck it on a hardened Windows host to re-enable the guard. |
| `ContainerExtension_AllowNativeFallback` | `false` | Fall back to native host execution if container daemon is offline. |
| `ContainerExtension_Platform` | `"auto"` | Overrides container architecture target (e.g., `linux/amd64`). |

## B.3 Runtime-Specific Configuration Manuals

### B.3.1 OrbStack (macOS)

OrbStack is a highly optimized, lightweight Docker Desktop alternative for macOS. It integrates natively with the macOS Hypervisor framework, offering superior I/O performance via VirtioFS.

1. **Socket Mapping:** OrbStack establishes a standard Unix domain socket at `/var/run/docker.sock`, which is automatically symlinked. If OneWare cannot locate the socket, set the configuration key:

   ```
   ContainerExtension_DaemonSocket = unix:///var/run/docker.sock
   ```

2. **Architecture Emulation (Apple Silicon):** When running on Apple Silicon (M-Series chip), running AMD64 images incurs a translation tax via Rosetta 2. For maximum performance, configure the platform override to native ARM64:

   ```
   ContainerExtension_Platform = linux/arm64
   ```

   Alternatively, keep it on `auto` to allow the extension to query the daemon and determine availability.

3. **Resource Optimization:** Ensure OrbStack's shared memory limit matches or exceeds the workspace requirements. If a synthesis run fails due to insufficient memory, set:

```
ContainerExtension_MemoryLimit = 8192 # Limits container to 8 GB
```

### B.3.2 Docker Desktop (Windows and macOS)

Docker Desktop is the official cross-platform container management system.

1. **Named Pipe Impersonation (Windows):** On Windows hosts, Docker Desktop communicates via named pipes, typically at `npipe://./pipe/docker_engine`. The client always opens the pipe at `TokenImpersonationLevel.Identification`, so the daemon endpoint can authenticate the caller but cannot impersonate its token. On top of this, an optional server-trust check verifies the pipe server process before the connection is used.

2. **Bypassing the Server-Trust Check:** That server-trust check is Windows-only and produces false positives on common configurations (WSL2 relays, rootless or remote engines), so it is *bypassed by default* (`ContainerExtension_BypassNamedPipeCheck=true`). On a hardened Windows host where the daemon is local and trusted, uncheck "Bypass Named Pipe Security Check" to re-enable it:

   ```
   ContainerExtension_BypassNamedPipeCheck = false
   ```

3. **WSL 2 Integration:** For Windows hosts, ensure WSL 2 is enabled in the Docker Desktop settings and that the target Linux distribution is ticked under **Resources -> WSL Integration** to ensure correct absolute directory path mappings.

### B.3.3 Podman (Rootless Linux Setup)

Podman is a daemonless, rootless container engine that operates natively on Linux.

1. **Rootless Socket Setup:** To run Podman without root privileges, enable the user-level systemd service:

   ```
   systemctl --user enable --now podman.socket
   ```

   This exposes a local user socket at `/run/user/$UID/podman/podman.sock`.

2. **Daemon Connection Config:** Configure the OneWare settings to point to the user-space socket:

   ```
   ContainerExtension_DaemonSocket = unix:///run/user/1000/podman/podman.sock
   ContainerExtension_DockerRuntimePath = /usr/bin/podman
   ```

   Replace `1000` with the target host user ID.

3. **User Namespace Mapping:** Rootless execution requires correct UID mapping. Verify that `/etc/subuid` and `/etc/subgid` have ranges defined for the execution user:

```
user:100000:65536
```

   This mapping translates host user ID `1000` inside the container boundary seamlessly, resolving host-to-container write permission issues on bound output directories.

## B.4 Troubleshooting Guide

### B.4.1 Socket Permissions (Linux)

If compilation fails with socket permission errors, ensure the current user belongs to the `docker` group:

```
sudo usermod -aG docker $USER
newgrp docker
```

If the group configuration is restricted by system administrators, apply temporary read-write permissions to the socket file:

```
sudo chmod 666 /var/run/docker.sock
```

### B.4.2 Dangling Container Reap

On start, the Container Extension scans for and reaps any orphaned containers matching the user-configured name prefix. If dangling containers are leaking memory or host resources, verify the prefix:

```
ContainerExtension_ContainerNamePrefix = containerextension-
```

Spawned containers are automatically reaped after run completion unless the configuration key `ContainerExtension_AutoRemove` is set to `false`.

# Appendix C

# Benchmark and Telemetry Logs

This appendix presents the empirical data logs captured during performance benchmarking and telemetry stress testing of the *OneWare Container Extension*.

Paths, timestamps, and host identifiers in the listings that follow reflect the repository layout and host configuration at the time of each run. Absolute user-home paths have been replaced with a neutral `<project>/` placeholder; the remaining values are reproduced as captured to preserve the integrity of the recorded evidence.

## C.1 Security Validation Output and Exceptions

This section contains the stack trace output captured when a command containing disallowed shell chaining characters is executed.

### C.1.1 Shell Injection Command Blocking Exception

When executing a shell command that attempts chaining via the `&&` operator, the security validation module intercepts the call and raises a `System.ArgumentException`. Below is the unhandled exception stderr log:

Listing C.1: Security Module Exception on Command Operator Check

```
Unhandled exception. System.ArgumentException: Nested shell expansion or command separators detected in argument: mkdir -p build && ghdl -i (Parameter 'command')
 at ContainerExtension.DockerExecutionStrategy.ExecuteAsync(ToolCommand command, CancellationToken cancellationToken) in <project>/src/ContainerExtension/DockerExecutionStrategy.cs:line 2525
 at ContainerBenchmarkHarness.Program.Main(String[] args) in <project>/src/ContainerBenchmarkHarness/Program.cs:line 81
 at ContainerBenchmarkHarness.Program.<Main>(String[] args)
```

## C.2 JSON-Lines Telemetry Stress-Test Logs

Below is the console output log captured during a multi-threaded concurrent stress test of the logging engine, demonstrating the system's resilience under heavy multi-instance telemetry load.

Listing C.2: ContainerTelemetry Multi-Threaded Stress Test Output Log

```
[Parent 58811] Spawning 3 child processes...
[Parent 58811] Starting telemetry stress test: 16 threads, 1000 iterations per thread.
[Parent 58811] Done in 2602ms. Success: 16000, Errors: 0
[Parent 58811] Waiting for 3 child processes to finish...
[Parent 58811] All child processes finished.
```